



Research article

Real-time intrusion detection in IoT networks: A hybrid framework of offline ML and online DL

Dusmurod Kilichev , Wooseong Kim *

Department of Computer Engineering, Gachon University, Seongnam, 13120, South Korea

ARTICLE INFO

2020 MSC:

68T07

68M25

Keywords:

Concept drift

Internet of things

Intrusion detection

Offline-online learning

Page-Hinkley

Probability calibration

Real-time

ABSTRACT

Internet of Things (IoT) intrusion detection must adapt to evolving traffic without relying on immediate ground-truth labels. This study proposes a label-free hybrid intrusion detection framework that integrates an offline RF teacher with an online Long Short-Term Memory student. The framework combines probability calibration, Mahalanobis-distance-based distribution monitoring, Page-Hinkley drift detection, entropy-based model selection, and confidence-gated pseudo-label adaptation. A leakage-resistant three-way protocol separates offline training, validation-time adaptation, and held-out testing, with labels used only for post-hoc evaluation. Experiments on CICIoMT2024 show that the proposed framework achieves 0.9893 accuracy, 0.8823 macro F1, and 0.9889 weighted F1 on the held-out multiclass stream. Under binary Benign-versus-Attack evaluation, it attains 0.9937 accuracy and 0.9257 macro F1, while attack recall reaches 0.9987. The framework also maintains strong ranking performance, low calibration error, and real-time processing, with a mean latency of 12.97 ms per sample and throughput of 77.13 samples per second. These results show that a strong offline detector can be extended with controlled on-line adaptation and label-free drift awareness while preserving high operational performance for gateway- and edge-level IoT monitoring.

1. Introduction

The Internet of Things (IoT) has enabled extensive connectivity, real-time monitoring, automation, and enhanced control across domains ranging from smart homes to industrial systems [1]. As IoT connections continue to increase worldwide (Fig. 1), they are creating a new paradigm of cyber-physical integration [2]. However, this rapid expansion has also introduced significant cybersecurity challenges [3]. IoT devices are highly heterogeneous and often have limited computational resources and minimal built-in security, making them attractive targets for attackers [4].

Protecting IoT networks therefore requires security mechanisms that are accurate, lightweight, adaptive, and suitable for changing traffic conditions [5]. Intrusion Detection Systems (IDSs) support this objective by monitoring network behaviour and identifying malicious activities that bypass preventive defences [6]. Because many IoT devices cannot support complex local security mechanisms, IDS-based monitoring provides an effective defence layer at the network, gateway, or edge level [7]. Machine learning (ML) and deep learning (DL) techniques have further improved IDS capabilities by detecting complex traffic patterns and attack signatures [8]. Nevertheless, dynamic IoT environments require IDS models that can recognise known attacks reliably while responding to evolving traffic and emerging attack variants [9].

* Corresponding author.

E-mail addresses: dusmurod@gachon.ac.kr (D. Kilichev), wooseong@gachon.ac.kr (W. Kim).

<https://doi.org/10.1016/j.iot.2026.102039>

Received 20 December 2024; Received in revised form 28 June 2026; Accepted 20 July 2026

Available online 21 July 2026

2542-6605/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

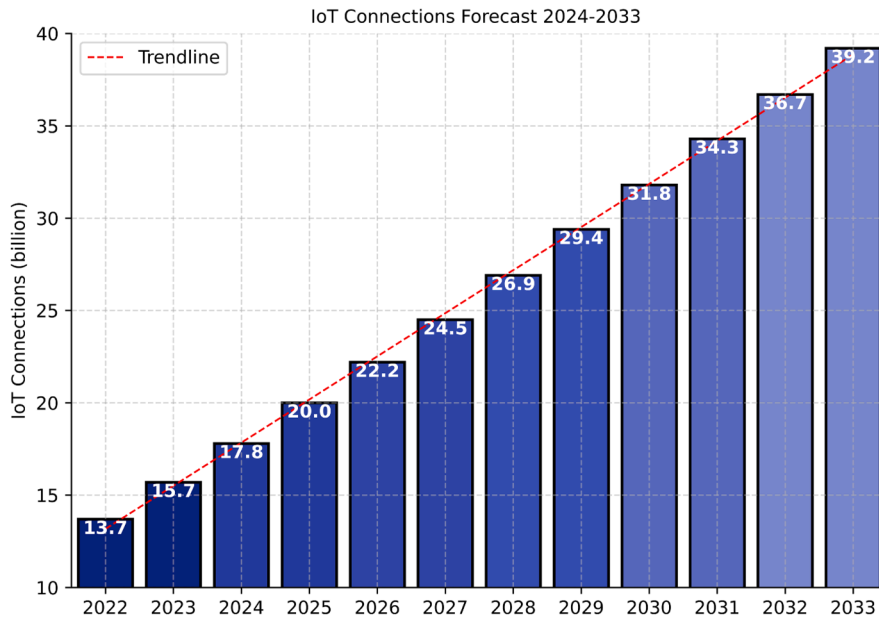


Fig. 1. Global IoT connections: 2022–2033.

Traditional offline learning-based IDS models are trained on fixed historical datasets [10]. They generally provide stable detection of known attacks and reliable performance when deployment traffic resembles the training distribution. However, they remain static after deployment. As new devices and services appear and attackers modify their behaviour, network-traffic distributions may change. Offline IDS models may therefore be affected by concept drift and respond poorly to new threats, particularly when frequent retraining is impractical.

Online learning-based IDS models address this limitation by updating their knowledge from streaming data [11]. However, streaming traffic may contain noise, incomplete observations, abrupt fluctuations, and misleading patterns that destabilise continuous learning [12]. Online models may also depend on uncertain pseudo-labels, accumulate errors, or begin with insufficient prior knowledge. In addition, continuous updating can increase computational, memory, and energy costs in resource-constrained IoT environments. Thus, online learning improves adaptability but may not provide sufficient stability for dependable intrusion detection.

These limitations motivate a hybrid offline–online IDS framework that combines offline stability with online adaptability. The offline component captures reliable patterns from historical normal and malicious traffic, providing a stable foundation for detecting known attacks. The online component complements this foundation by adapting incrementally to evolving traffic conditions and emerging attack variants. The framework therefore seeks to balance robustness and responsiveness in dynamic IoT environments.

Despite progress in ML- and DL-based IDSs, a gap remains in developing a resource-efficient approach that preserves reliable offline knowledge while adapting incrementally to changing IoT traffic. To address this gap, this study proposes a hybrid IDS framework that integrates an offline ML model with an online DL component for real-time intrusion detection. The offline component provides a stable detection foundation, while the online component supports adaptation to emerging traffic patterns and attack variants.

The main contributions of this study are summarised as follows:

- **Label-free hybrid IDS for streaming IoT traffic.** A hybrid intrusion detection framework is developed by integrating an offline Random Forest (RF) teacher with an online Long Short-Term Memory (LSTM) student. The framework processes unlabelled traffic streams without requiring real-time ground-truth labels during deployment.
- **Unsupervised concept-drift monitoring.** A label-independent drift-detection mechanism is introduced using Ledoit–Wolf covariance estimation, Mahalanobis distance, and the Page–Hinkley sequential change-point test. This mechanism identifies distributional changes without using prediction errors or delayed labels.
- **Entropy-based dynamic model selection.** A confidence-aware switching strategy dynamically selects between RF and LSTM predictions using the Shannon entropy of their probability outputs. This enables label-free model selection during streaming inference.
- **Confidence-gated pseudo-label adaptation.** The online LSTM is incrementally updated using RF-generated pseudo-labels only when predefined confidence, entropy, margin, and in-distribution criteria are satisfied. This selective adaptation reduces the risk of propagating uncertain pseudo-labels.
- **Leakage-resistant streaming evaluation.** A strict three-way partitioning protocol separates offline training, validation-time adaptation, and held-out testing. Ground-truth labels are withheld from the streaming loop and used exclusively for post-hoc evaluation, thereby limiting label leakage and more closely representing practical IDS deployment.

- **Comprehensive evaluation of streaming IDS behaviour.** The framework is evaluated using aggregate and class-wise classification metrics, confusion matrices, ROC and precision-recall analysis, calibration measures, IDS-specific operational rates, temporal adaptation and forgetting analysis, label-free diagnostics, threshold sensitivity, misclassification analysis, and computational efficiency.

For evaluation, we test our approach on the CICIoMT2024 dataset—a comprehensive, real-time benchmark specifically designed for multi-protocol security assessment in IoT environments. Our framework is publicly available in a GitHub repository¹, reflecting our commitment to transparency and encouraging the research community to contribute to ongoing advancements in this area.

The remainder of this paper is organised as follows. Section 2 reviews related work, Section 3 presents the proposed RF-LSTM framework, and Section 4 describes the experimental setup. Section 5 reports the results, Section 6 discusses the findings, and Section 7 concludes the study.

2. Related work

Research on IoT and Industrial IoT (IIoT) intrusion detection has progressively moved from static offline classification toward adaptive, streaming, and continual-learning frameworks. Early ML IDSs typically relied on offline training over labelled benchmark traffic, where models such as decision trees, RFs, support vector machines (SVMs), neural networks, and autoencoders were trained once and evaluated using conventional train-test protocols. Although these methods often achieve high accuracy on known attacks, they are limited by their dependence on representative labelled data and their inability to adapt when traffic distributions or attack behaviours evolve. This limitation is especially critical in IoT/IIoT environments, where heterogeneous devices, changing communication patterns, and newly emerging attacks can rapidly invalidate a static model.

A first group of works addresses this problem through hybrid offline-online learning. Odiathevar et al. [13] proposed a hybrid anomaly detector using an offline Radius Nearest Neighbour model and an online SVM. The offline component classified known traffic and supplied confidence scores, while the online SVM adapted using high-confidence samples. This work is important because it explicitly separates stable offline knowledge from adaptive online learning. However, concept drift is handled only implicitly through online boundary updates, and the evaluation remains dataset-based rather than live. Odiathevar et al. [14] extended this idea with offline autoencoder-based representation learning and online outlier detection using IOCSVM, IGMM, OCSVM, GMM, LOF, and KDE. Unlike many hybrid systems, it included a one-sided Mann-Whitney U test over reconstruction-error distributions to control threshold drift. This provides a stronger drift-aware mechanism, although it is still not a general-purpose stream drift detector such as ADWIN or Page-Hinkley.

Several studies use online or incremental learning for real-time or near-real-time IDS operation, but with varying degrees of practical deployment realism. Adewole et al. [15] compared batch learners, J48 and PART, with stream learners, Hoeffding Tree and OzaBagAdwin. Their findings support the suitability of data-streaming models for scalable intrusion detection, particularly in binary classification. However, the study is primarily empirical and does not deeply analyse drift signals, although OzaBagAdwin includes ADWIN-based adaptation. Gelenbe and Nakip [16] proposed AADRNN for real-time IoT cyberattack detection. Their method supports offline and online learning from normal traffic and is directly relevant to real-time IoT detection. Nevertheless, no explicit drift detector is implemented, and online updates assume access to legitimate normal traffic without fully specifying how contaminated updates are avoided. Agbedanu et al. [17] proposed ALMANET, a lightweight online IDS combining ALMA, stochastic weight averaging, and an incremental neural network. It processes samples one at a time and reports latency, throughput, and memory usage, making it one of the stronger real-time-oriented studies. Its main limitation is that online learning requires labelled stream samples, while drift is evaluated synthetically rather than detected operationally.

Another important direction is adaptive incremental learning for new attacks and evolving environments. Sun et al. [18] proposed an active incremental IDS for IIoT using a Multi-Stacking ensemble and active query mechanisms. The method combines known-attack classification with active learning for uncertain samples and improved KNN for unknown attacks. Its strength lies in addressing unknown attacks and incremental adaptation in an IIoT context, but it depends on oracle-labelled feedback and lacks explicit drift detection. Abderrahim and Benosman [19] combined offline LightGBM with an incremental Passive-Aggressive classifier. The LightGBM model provides batch-trained probability features, while the Passive-Aggressive classifier adapts through labelled mini-batches and dynamic regularisation. This hybrid design is relevant to IoT intrusion detection, yet it still requires labelled updates and leaves DDM/ADWIN-style drift detection to future work. Lu et al. [20] proposed AdaOIL-IDS for smart-grid intrusion detection using a Class-Correlated Broad Learning System. The model supports online incremental learning for new and old attack classes and uses a dynamic real-time loss evaluator to trigger offline retraining. This provides a custom drift-response mechanism, although it relies on labelled incoming data and is evaluated through simulated incremental experiments rather than live smart-grid deployment.

A substantial body of recent work focuses on class-incremental, few-shot, and continual learning. These studies are highly relevant to emerging attack detection because they attempt to learn new attack classes without full retraining. Cerasuolo et al. [21] applied class-incremental learning to IoT traffic using a 2D-CNN and methods such as fine-tuning, memory rehearsal, Learning without Forgetting, iCaRL+, and Bias Correction. Di Monda et al. [22] extended this line through few-shot class-incremental learning, identifying Rethinking Few-Shot as an effective approach for adapting to new attacks with limited samples. Du et al. [23] proposed BFS-NID, which uses a self-supervised Vision Transformer feature extractor and branch fusion strategy. Cerasuolo et al. [24] further studied adaptable, incremental, and explainable IoT network IDSs using CNN-based class and domain incremental learning. These works

¹ <https://github.com/TATU-hacker/Real-Time-Intrusion-Detection-in-IoT-Networks.git>

are valuable for handling emerging attack classes and catastrophic forgetting. However, they generally assume labelled new-class samples, operate in dataset-based adaptation sessions, and do not implement explicit concept drift detectors. Therefore, they address class emergence more than real-time unsupervised drift in streaming IoT traffic.

Domain adaptation and continual replay methods address related but distinct problems. Violos et al. [25] proposed OLIDA-IDS, combining an offline RF, unsupervised domain adaptation, and an online Adaptive RF. The work is useful because it bridges offline supervised learning, target-domain adaptation, and online learning. However, its drift handling is inherited from the online forest model and not specified as a standalone detector, while the labelling or pseudo-labelling mechanism for target traffic is not fully detailed. Fathima et al. [26] proposed TAMR, a task-aware memory replay framework for continual-learning network IDS.

Table 1

Comparative analysis of related works on offline, online, adaptive, and drift-aware IDS methods for IoT/IoT and network intrusion detection.

Study	Domain Dataset(s)	and	Offline Learning	Online/Incremental Learning	Drift Detection	Real-Time Support	Key Limitations	Comparison with Proposed Work
[13]	Network anomaly detection; NSL-KDD		Yes; Rad-NN trained offline on labelled data	Online SVM; batch updates using high-confidence samples	Implicit only; no named detector	Partial	No explicit drift detector; dataset-based evaluation	Proposed method adds explicit Page-Hinkley drift detection and label-free pseudo-label gating
[14]	Streaming network anomaly detection; UNSW-NB15, CTU-13		Yes; AE, VAE, DAE trained on normal traffic	Online outlier models; IOCSVM, IGMM, OCSVM, GMM, LOF, KDE	Mann-Whitney U test on reconstruction-error distribution	Partial	Assumes normal dominance; benchmark stream evaluation	Proposed method combines supervised RF teacher with unsupervised Mahalanobis drift monitoring
[15]	Network IDS; UNSW-NB15		Yes; J48 and PART batch models	HT and OzaBagAdwin using test-then-train stream evaluation	Partial; ADWIN through OzaBagAdwin	Partial	Stream labels required; limited drift analysis	Proposed method supports label-free stream adaptation and explicit drift monitoring
[18]	IIoT IDS; Edge-IIoTset		Yes; Multi-Stacking ensemble	Active incremental learning with committee query and improved KNN	Implicit; no named drift detector	Partial	Requires oracle labels; no explicit drift detector	Proposed method avoids oracle feedback and uses Page-Hinkley drift detection
[32]	IIoT multivariate time-series anomaly detection; KDD99, SMD, MSL, SMAP, others		Yes; TCN, VAE, PNF	Block-based online feedback retraining	Heuristic anomaly inversion	Partial	Not primarily network IDS; heuristic drift response	Proposed method targets IoT network IDS and uses formal drift detection
[16]	Real-time IoT cyber-attack detection; Kitsune, KDD99, MedBioT, Bot-IoT		Yes; AADRNN trained on normal traffic	Incremental AADRNN updates over normal windows	No explicit detector	Yes	Assumes availability of clean normal traffic; contamination risk	Proposed method adds gated pseudo-labelling and unsupervised drift monitoring
[21]	IoT IDS; IoT-23		Yes; 2D-CNN trained on known classes	Class-incremental learning for labelled new attacks	No	Partial	Requires labelled new attack samples; no stream learning	Proposed method adapts during streaming without using ground-truth stream labels
[22]	Network and IoT network IDS; CSE-CIC-IDS2018, IoT-NID		Yes; CNN with RFS-based FSCIL	Few-shot class-incremental adaptation	No	Partial	Requires labelled few-shot samples; no drift detector	Proposed method provides online pseudo-labelled adaptation and explicit drift detection
[23]	Network IDS; CIC-IDS2017, CIC-IDS2018	CSE-	Yes; self-supervised ViT feature extractor	FSCIL with branch classifier learning and branch fusion	No	Partial	High complexity; labelled new classes required	Proposed method uses drift-aware pseudo-label learning rather than labelled FSCIL sessions
[20]	Smart-grid IDS; IEC60870-5-104 datasets		Yes; CC-BLS trained on labelled normal and attack traffic	CC-BLS online incremental learning with labelled blocks	Dynamic loss-threshold evaluator	Partial	Requires labelled online data; dataset-based incremental evaluation	Proposed method does not require stream labels and uses unsupervised drift signal
[17]	Real-time IoT IDS; NF Bot-IoT, NF ToN-IoT, NF UNSW, NF CSE2018		No; purely online	ALMANET; sample-wise labelled online updates	Drift robustness evaluation only	Yes	Requires labelled online samples; no explicit detector	Proposed method includes offline teacher, label-free online adaptation, and drift detector
[28]	QUIC IDS; custom QUIC dataset, CESNET-QUIC22		Partial; small pre-training set	ARF prequential learning with feature selection	ADWIN on classifier error rate	Partial	Requires labelled flows; QUIC-specific attacks	Proposed method uses unsupervised drift signal and IoT-focused dataset
[33]	IoT zero-day detection; simulated IoT graph data		Partial; GNN training not clearly separated	Mini-batch GNN on-line updates	No	Partial	Simulated data; label acquisition unclear; no drift detector	Proposed method provides clearer streaming protocol and explicit drift mechanism
[27]	Online network IDS; NSL-KDD, UNSW-NB15		Yes; AE with QHCL and GMM	Pseudo-labelled mini-batch online updates	Implicit only; no named detector	Partial	Not IoT-specific; no formal drift detector	Proposed method uses IoTMT data and Page-Hinkley drift detection
[25]	Domain-adaptive IDS; CIC-IDS2017, CIC-IDS2018, PS-Azure2023		Yes; offline RF with sMDA adaptation	ARF online learning in target domain	Partial; ARF internal drift handling	Partial	Label or pseudo-label mechanism unclear; no standalone detector	Proposed method defines explicit pseudo-label gates and drift signal
[26]	Continual network IDS; KDD99, NSL-KDD, CIC-IDS2017/2018, Bot-IoT		Partial; SimpleDNN initial model	TAMR memory replay with labelled tasks	Drift-aware replay; no standalone detector	Partial	Requires labelled task data and task IDs	Proposed method avoids task IDs and uses unsupervised drift monitoring
[19]	IoT IDS; CICIoT2023		Yes; LightGBM batch model	Passive-Aggressive mini-batch incremental learning	No; DDM/ADWIN left for future work	Partial	Requires labelled mini-batches; no explicit drift detector	Proposed method uses pseudo-labels and Page-Hinkley drift detection
[24]	IoT network IDS; IoT-NID, TON IoT, Edge-IIoT		Yes; 2D-CNN	CIL/DIL with labelled target data	No	Partial	Dataset-based adaptation; no live drift detection	Proposed method performs streaming adaptation with logged drift events
[31]	CTI-enhanced IDS; NSL-KDD, UNSW-NB15, simulated CTI		Yes; SVM, K-Means, GRU/LSTM	Event-driven CTI-triggered updates	Implicit CTI-triggered adaptation	Partial	CTI construction, labels, and drift mechanism underspecified	Proposed method gives explicit pseudo-label and drift criteria
[30]	Honeypot web IDS; Apache logs, honeypot HTTP logs		Yes; MultinomialNB supervised warm-up	River-based pseudo-label incremental updates	No	Partial	HTTP-log focus; no explicit drift detector	Proposed method targets IoTMT network traffic and monitors drift explicitly
[29]	Edge/IoT IDS; CIC-IDS2017, UNSW-NB15, IoTID20		Yes; RF deployed at edge	No final online learning; real-time inference only	No	Yes	Static deployed model; no online adaptation or drift detector	Proposed method adds online LSTM adaptation and drift detection
This work	Real-time IoT/IoMT IDS; CICIoMT2024		Yes; calibrated RF teacher	Online LSTM student with gated RF pseudo-labels	Page-Hinkley over Mahalanobis distance	Yes	Simulated streaming; no live IoT deployment	Integrates offline teacher, label-free online adaptation, and explicit unsupervised drift detection

TAMR prioritises replay using similarity, rarity, and recency, improving retention under evolving tasks. Its main contribution is drift-aware continual replay rather than explicit drift detection; it still relies on labelled task data and task segmentation. Liu et al. [27] proposed QHCL-AOC-IDS, an uncertainty-aware online IDS using an autoencoder, quadruplet-hard contrastive learning, pseudo-labelling, and GMM-based open-set detection. It is strong in online adaptation and zero-day rejection, but it is not IoT-specific and does not implement a named drift detector.

Several works focus on specialised deployment contexts. Alamani et al. [28] introduced QUIC-AID for encrypted transport intrusion detection using Adaptive RF, ADWIN, and dynamic feature selection. This is one of the clearest examples of explicit drift detection, as ADWIN monitors classifier error and triggers tree replacement. However, it depends on labelled prequential evaluation and is specific to a narrow traffic protocol and attack setting. Kumar et al. [29] proposed an edge/IoT IDS using an offline-trained RF deployed for real-time inference. It reports strong latency and resource results, but the deployed system is static: its “online” component refers to real-time inference rather than online learning. Alkamli et al. [30] proposed a honeypot-driven continuous-learning platform using River-based Multinomial Naive Bayes and confidence-filtered pseudo-labels from HTTP honeypot logs. Although practically interesting, it is web-log focused rather than IoT network-flow focused and lacks explicit drift detection. Reddy et al. [31] proposed DICI + +, a CTI-enhanced adaptive IDS using SVM, K-Means, GRU/LSTM, and event-driven updates. Its CTI-driven design is promising, but the online update protocol, labelling source, threat-intelligence simulation, and drift mechanism are underspecified.

A smaller group of works investigates broader anomaly detection or graph-based methods. Fan et al. [32] proposed OOA-UADS for IIoT multivariate time-series anomaly detection using TCN, VAE, PNF, POT thresholding, and anomaly inversion. It is relevant to streaming IIoT anomaly detection and concept-drift adaptation, but it is not primarily an IoT network intrusion detection method and its drift response is heuristic. Dhanush et al. [33] proposed a GNN-based IoT zero-day detector with online mini-batch updates on simulated IoT traffic. While graph modelling is promising for IoT networks, the offline and online protocols are underspecified, labels appear necessary, and no explicit drift detector is provided.

Across these studies, several gaps remain (Table 1). First, many adaptive IDSs require labelled online data, oracle feedback, labelled new-class samples, or task IDs. This is difficult to satisfy in real IoT/IIoT deployments, where attacks may be unknown and immediate ground truth is rarely available. Second, many papers conflate online adaptation with drift detection: models are updated incrementally, but the system does not explicitly monitor a drift signal or define a drift-triggered response. Third, real-time claims are often supported by dataset replay, prequential evaluation, or static inference latency rather than live IoT deployment with online adaptation. Fourth, few works combine a strong offline model for known attack detection, label-free online adaptation, explicit unsupervised drift monitoring, and conservative pseudo-label filtering in a single framework.

The proposed RF-LSTM hybrid IDS addresses these gaps by combining a calibrated offline RF teacher with an online LSTM student and explicit Page-Hinkley drift detection over Mahalanobis distance. Unlike labelled stream-learning approaches, the online student adapts from RF-generated pseudo-labels accepted only when confidence, entropy, margin, and in-distribution gates are satisfied. Unlike implicit adaptation methods, the proposed system monitors an unsupervised distributional signal through a Ledoit-Wolf training-distribution fingerprint and Page-Hinkley detection. Compared with static real-time IDSs, it supports streaming adaptation rather than inference alone. Although its evaluation is still based on simulated streaming rather than live IoT deployment, it more directly integrates offline learning, label-free online adaptation, and explicit drift detection than most reviewed works.

3. Proposed hybrid framework

3.1. System overview

This study proposes a hybrid IDS for real-time IoT network monitoring that combines the stability of offline ML with the adaptability of online DL. The framework consists of three main components: a calibrated RF teacher deployed on the monitoring server, an incrementally updated LSTM student deployed on the IoT gateway (Fig. 2), and an unsupervised distributional shift monitoring module. The RF teacher provides stable predictions on labelled historical traffic, while the LSTM student adapts during deployment using carefully filtered pseudo-labels. The system processes network traffic sequentially, sample by sample, under realistic conditions where ground-truth labels are unavailable during operation (Table 2).

During deployment, each incoming sample is processed by both the calibrated RF teacher and the LSTM student. The RF output is used to generate pseudo-labels only when four acceptance conditions are simultaneously satisfied: high confidence, low entropy, sufficient top-class margin, and in-distribution membership. Accepted pseudo-labels are stored in a replay buffer and used to update the LSTM incrementally. The final prediction is selected through a composite confidence-based switching mechanism, with hysteresis applied to prevent unstable alternation between RF and LSTM outputs. In parallel, distributional shift is monitored using the Page-Hinkley test applied to Mahalanobis distances derived from the training-data covariance fingerprint (Algorithm 1).

The framework follows three methodological principles. First, data leakage is prevented by computing all labelled-data artefacts, including preprocessing statistics, calibration parameters, and the covariance fingerprint, only from the offline training split before deployment. Second, the streaming phase remains label-free: true labels are not accessed during inference, adaptation, or shift monitoring, and are used only for post-hoc evaluation. Third, the system distinguishes distributional shift from concept drift.

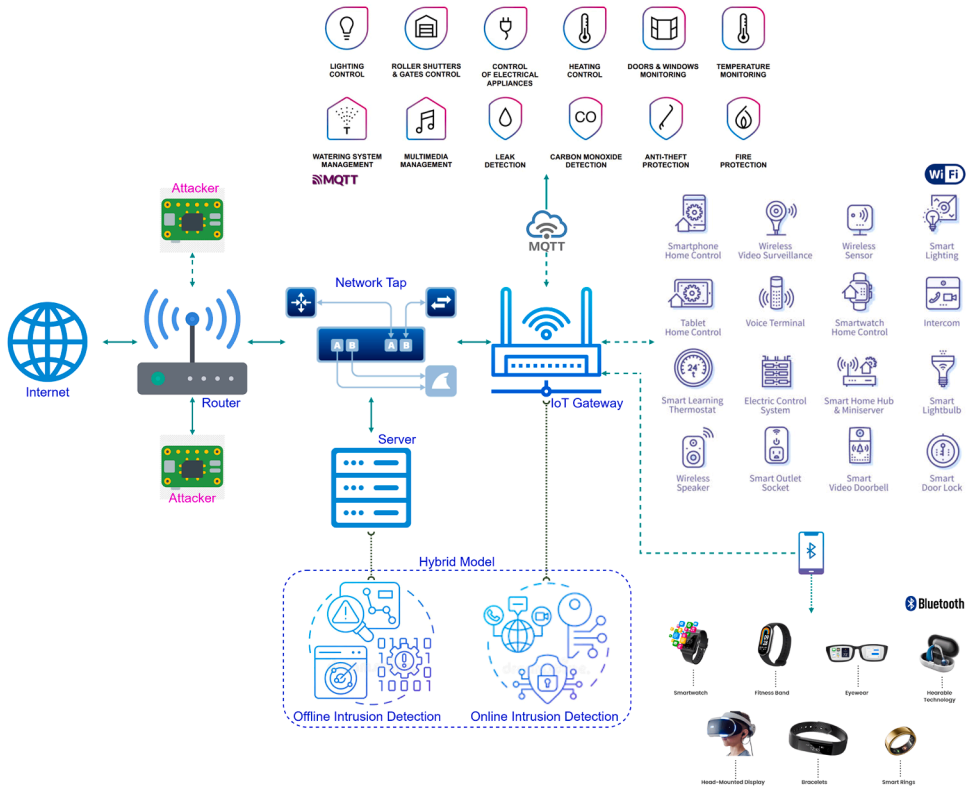


Fig. 2. Hybrid intrusion detection: teacher on server (ML), student on gateway (DL).

Table 2

Main components and operational roles of the proposed hybrid IDS.

Component	Role
RF teacher	Provides calibrated offline predictions and pseudo-label candidates.
LSTM student	Adapts incrementally from accepted pseudo-labels during deployment.
Shift monitor	Detects distributional changes in the input feature space.
Hybrid switch	Selects the final prediction source at each time step.
Post-hoc evaluator	Computes supervised metrics after the streaming phase ends.

Algorithm 1 Sample-wise operation of the proposed hybrid IDS.

```

1: for each incoming network sample  $x_t$  do
2:   Obtain RF and LSTM probability outputs.
3:   Evaluate the pseudo-label acceptance conditions.
4:   if the pseudo-label is accepted then
5:     Store the pseudo-labelled sample in the replay buffer.
6:     Incrementally update the LSTM student.
7:   end if
8:   Select the final prediction using the hybrid switching rule.
9:   Store predictions and diagnostic information for post-hoc evaluation.
10: end for

```

3.2. Offline RF teacher with probability calibration

The offline component provides a stable teacher model before the proposed IDS enters the label-free deployment phase. The labelled offline data are stratified into an RF-fit set, representing 80% of the offline pool, and a calibration set, representing the remaining 20%. The RF-fit set is used to train the RF teacher and estimate feature standardisation parameters, whereas the calibration set is reserved exclusively for probability calibration. Once estimated, the preprocessing parameters are frozen and applied unchanged

Table 3
Configuration of the offline RF teacher model.

Component	Configuration
Model type	RF classifier
Number of trees	150
Maximum depth	None
Minimum samples per leaf	1
Class weight	balanced_subsample
Probability calibration	Platt-sigmoid (CalibratedClassifierCV)
Calibration ratio	0.20 (20% offline data)
Training/calibration split	80% RF-fit, 20% calibration
Data scaling	StandardScaler fitted on RF-fit only

to all subsequent data, including calibration and deployment samples, thereby preventing leakage from held-out or streaming traffic (Table 3).

The RF teacher is trained using 150 decision trees, unrestricted tree depth, class-balanced bootstrap weighting, and a minimum terminal-node size of one sample. The class-balanced weighting mechanism reduces the effect of severe class imbalance by adjusting class contributions within each bootstrap sample, without relying on explicit oversampling. In the proposed framework, the RF teacher serves two main roles: it provides stable offline classification and generates probabilistic guidance for pseudo-label-based adaptation of the online LSTM student.

Because raw ensemble probabilities may be overconfident, post-hoc Platt-sigmoid calibration is applied using the held-out calibration set. This step improves the reliability of class-probability estimates without retraining the forest. Calibrated probabilities are essential because the pseudo-label gate, entropy and margin criteria, and hybrid switching function all depend on trustworthy confidence estimates. Therefore, the calibrated RF replaces the uncalibrated model for all deployment-time probability queries (Algorithm 2).

Algorithm 2 Offline RF teacher construction.

Require: Labelled offline dataset D_{off}

Ensure: Calibrated RF teacher M_{RF}^{cal}

- 1: Stratify D_{off} into RF-fit set D_{fit} and calibration set D_{cal}
 - 2: Estimate preprocessing statistics using D_{fit} only
 - 3: Freeze the preprocessing parameters
 - 4: Train RF teacher M_{RF} on D_{fit}
 - 5: Apply frozen preprocessing to D_{cal}
 - 6: Calibrate M_{RF} using D_{cal}
 - 7: Freeze the calibrated teacher M_{RF}^{cal} for deployment
-

3.3. Mahalanobis distribution fingerprint and drift detection

To support label-free monitoring during deployment, the proposed framework constructs a distributional fingerprint from the RF-fit training data. For each scaled feature vector $\mathbf{x} \in \mathbb{R}^p$, its deviation from the training distribution is measured using the squared Mahalanobis distance [34]:

$$D_M^2(\mathbf{x}) = (\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}), \quad (1)$$

where $\boldsymbol{\mu}$ denotes the mean vector and $\boldsymbol{\Sigma}^{-1}$ is the inverse covariance matrix estimated from the RF-fit data. Unlike independent feature-wise distance measures, the Mahalanobis distance accounts for correlations among features, making it suitable for heterogeneous IoT traffic representations. Since direct covariance inversion can be unstable in high-dimensional settings, a Ledoit–Wolf shrinkage estimator is used to obtain a well-conditioned covariance fingerprint [35].

The in-distribution boundary is defined empirically as the 97.5th percentile of the training Mahalanobis distances:

$$\tau_{id} = \text{Quantile}_{0.975} \{ D_M^2(\mathbf{x}_i) \}_{i=1}^{n_{\text{train}}}. \quad (2)$$

A deployment sample is treated as out-of-distribution when $D_M^2(\mathbf{x}) > \tau_{id}$. In this study, the feature dimension is $p = 45$, and the empirical threshold is $\tau_{id} = 166.863$. This empirical threshold is used as the primary gate instead of the theoretical χ^2 reference threshold of 65.410, because IoT network traffic rarely follows an ideal multivariate Gaussian distribution. The training Mahalanobis statistics further support this choice, with a median of 4.85, a 95th percentile of 91.53, and a maximum of 435370.20, indicating a highly skewed distance distribution. These statistics are summarised in Table 4.

During deployment, distributional shift is monitored by applying the Page–Hinkley test to the stream of per-sample Mahalanobis distances [36]. The detector maintains an online running mean $\hat{\mu}_t$ and cumulative deviation score S_t :

$$\hat{\mu}_t = \hat{\mu}_{t-1} + \frac{D_M^2(\mathbf{x}_t) - \hat{\mu}_{t-1}}{t}, \quad (3)$$

Table 4
Distribution fingerprint statistics used for Mahalanobis-based monitoring.

Statistic	Value
Covariance estimator	Ledoit–Wolf
Feature dimension, p	45
Empirical threshold, τ_{id} ($q = 0.975$)	166.863
χ^2 reference threshold	65.410
Training D_M^2 median	4.85
Training D_M^2 95th percentile	91.53
Training D_M^2 maximum	435370.20

$$S_t = \max(0, S_{t-1} + D_M^2(\mathbf{x}_t) - \hat{\mu}_t - \delta), \quad (4)$$

where $\delta = 0.005$ controls sensitivity to small sustained deviations. A drift alarm is raised only when the following three conditions are jointly satisfied:

$$t \geq t_{\min}, \quad S_t > \lambda, \quad D_M^2(\mathbf{x}_t) > \tau_{id}, \quad (5)$$

with $t_{\min} = 50$ and $\lambda = 50$. After an alarm, the detector state is reset and the event index is stored for post-hoc analysis. Algorithm 3 summarises the monitoring procedure.

Algorithm 3 Mahalanobis-based distribution shift monitoring.

Require: Scaled sample $\mathbf{x}_t$, training fingerprint (μ, Σ^{-1}) , threshold τ_{id}

- 1: Compute $D_M^2(\mathbf{x}_t)$ using Eq. (1)
 - 2: Mark $\mathbf{x}_t$ as out-of-distribution if $D_M^2(\mathbf{x}_t) > \tau_{id}$
 - 3: Update running mean $\hat{\mu}_t$ using Eq. (3)
 - 4: Update Page–Hinkley score S_t using Eq. (4)
 - 5: **if** $t \geq t_{\min}$ and $S_t > \lambda$ and $D_M^2(\mathbf{x}_t) > \tau_{id}$ **then**
 - 6: Raise distributional shift alarm
 - 7: Log event index t
 - 8: Reset detector state
 - 9: **end if**
-

3.4. Online LSTM student and pseudo-label adaptation

3.4.1. LSTM architecture

The online student model is implemented as a single-layer LSTM network designed for incremental adaptation during deployment. The model contains 32 hidden units, followed by a rectified linear activation layer and a linear classification head that produces logits over the six traffic classes. For each incoming sample, the student generates a class-probability estimate and, when a pseudo-label is accepted by the teacher-gated adaptation mechanism, updates its parameters through a single-sample gradient step. The optimisation process uses the AdamW algorithm with a learning rate of $\eta = 10^{-3}$, weight decay of $\lambda_w = 10^{-5}$, and cross-entropy loss. Table 5 summarises the configuration of the online LSTM student.

Table 5
Configuration of the online LSTM student model.

Component	Configuration
Student model	Single-layer LSTM
Hidden dimension	32 units
Activation function	ReLU
Output layer	Linear classification head
Number of classes	6
Optimiser	AdamW
Learning rate	$\eta = 10^{-3}$
Weight decay	$\lambda_w = 10^{-5}$
Loss function	Cross-entropy
Update mode	Single-sample incremental update

A key feature of the online student is its lightweight configuration, which makes it suitable for gateway-side inference and adaptation in resource-constrained IoT environments. Unlike the offline RF teacher, which remains fixed after training and calibration, the LSTM student is updated during the streaming phase using only accepted pseudo-labels. This enables the student to gradually adjust

to deployment traffic while avoiding direct access to ground-truth labels. In the hybrid architecture, the LSTM is therefore not used as an independent replacement for the RF teacher, but as an adaptive companion whose predictions are considered by the composite switching mechanism (Algorithm 4).

Algorithm 4 Online LSTM student update.

Require: Incoming sample $\mathbf{x}_t$, RF pseudo-label $\hat{y}_{\text{RF}}$, acceptance gate decision

- 1: Generate LSTM probability estimate $\mathbf{p}_{\text{LSTM}}(\mathbf{x}_t)$
 - 2: **if** the RF pseudo-label satisfies the acceptance gate **then**
 - 3: Add $(\mathbf{x}_t, \hat{y}_{\text{RF}})$ to the replay buffer
 - 4: Draw one pseudo-labelled sample from the replay buffer
 - 5: Update the LSTM parameters using one gradient step
 - 6: **else**
 - 7: Keep the LSTM parameters unchanged
 - 8: **end if**
 - 9: Store the LSTM output for hybrid switching and post-hoc analysis
-

3.4.2. Pseudo-label gate

The RF teacher generates pseudo-labels for online LSTM adaptation by selecting the class with the highest calibrated probability:

$$\hat{y}_{\text{RF}} = \arg \max_k p_{\text{RF},k}. \quad (6)$$

However, incorrect pseudo-labels may transfer teacher errors to the student model and degrade online adaptation. To reduce this risk, the proposed framework applies a strict acceptance gate before any pseudo-labelled sample is admitted to the replay buffer. The gate is deliberately conservative and consists of four conjunctive conditions: calibrated confidence, normalised entropy, top-2 probability margin, and in-distribution membership. A pseudo-label is accepted only when all four conditions are satisfied.

The first condition requires the RF teacher to assign sufficiently high confidence to its predicted class:

$$\max_k p_{\text{RF},k} \geq \tau_{\text{conf}} = 0.90. \quad (7)$$

The second condition limits predictive uncertainty by requiring the normalised Shannon entropy of the RF probability vector to remain below a fixed threshold [37]:

$$\tilde{H}(\mathbf{p}_{\text{RF}}) = \frac{-\sum_{k=1}^K p_{\text{RF},k} \log p_{\text{RF},k}}{\log K} \leq \tau_H = 0.35, \quad (8)$$

where $K = 6$ denotes the number of traffic classes. The third condition requires sufficient separation between the two most likely classes:

$$\Delta(\mathbf{p}_{\text{RF}}) = p_{\text{RF}}^{(1)} - p_{\text{RF}}^{(2)} \geq \tau_{\text{margin}} = 0.20, \quad (9)$$

where $p_{\text{RF}}^{(1)}$ and $p_{\text{RF}}^{(2)}$ denote the largest and second-largest class probabilities, respectively. This condition rejects ambiguous samples in which the predicted class is only weakly separated from its nearest competitor.

Algorithm 5 Pseudo-label acceptance gate.

Require: RF probability vector $\mathbf{p}_{\text{RF}}$, sample $\mathbf{x}$, Mahalanobis threshold τ_{id}

- 1: Compute $\hat{y}_{\text{RF}} \leftarrow \arg \max_k p_{\text{RF},k}$
 - 2: Compute confidence $\max_k p_{\text{RF},k}$
 - 3: Compute normalised entropy $\tilde{H}(\mathbf{p}_{\text{RF}})$
 - 4: Compute top-2 probability margin $\Delta(\mathbf{p}_{\text{RF}})$
 - 5: Compute Mahalanobis distance $D_M^2(\mathbf{x})$
 - 6: **if** $\max_k p_{\text{RF},k} \geq 0.90$ **and** $\tilde{H}(\mathbf{p}_{\text{RF}}) \leq 0.35$ **and** $\Delta(\mathbf{p}_{\text{RF}}) \geq 0.20$ **and** $D_M^2(\mathbf{x}) \leq \tau_{\text{id}}$ **then**
 - 7: Accept pseudo-label $(\mathbf{x}, \hat{y}_{\text{RF}})$
 - 8: Store accepted pair in replay buffer
 - 9: **else**
 - 10: Reject pseudo-label
 - 11: Skip LSTM update for the current step
 - 12: **end if**
-

The fourth condition requires the sample to remain within the training distribution according to the Mahalanobis-based distribution fingerprint:

$$D_M^2(\mathbf{x}) \leq \tau_{\text{id}}. \quad (10)$$

Thus, even high-confidence RF predictions are rejected if the corresponding sample is out-of-distribution, since such samples may represent unseen traffic patterns or novel attack behaviours for which the RF teacher cannot be assumed reliable. Formally, the pseudo-label acceptance rule is defined as:

$$\mathcal{A}(\mathbf{x}) = \mathbb{1} \left[\max_k p_{\text{RF},k} \geq \tau_{\text{conf}} \wedge \tilde{H}(\mathbf{p}_{\text{RF}}) \leq \tau_H \wedge \Delta(\mathbf{p}_{\text{RF}}) \geq \tau_{\text{margin}} \wedge D_M^2(\mathbf{x}) \leq \tau_{\text{id}} \right]. \quad (11)$$

If $\mathcal{A}(\mathbf{x}) = 1$, the pair $(\mathbf{x}, \hat{y}_{\text{RF}})$ is stored in the replay buffer and may be used for LSTM adaptation. If any condition fails, the sample is discarded and no gradient update is issued at that time step (Algorithm 5). Table 6 summarises the acceptance criteria.

Table 6

Pseudo-label acceptance gate and threshold values.

Criterion	Condition	Threshold
Calibrated confidence	$\max_k p_{\text{RF},k} \geq \tau_{\text{conf}}$	$\tau_{\text{conf}} = 0.90$
Normalised entropy	$\tilde{H}(\mathbf{p}_{\text{RF}}) \leq \tau_H$	$\tau_H = 0.35$
Top-2 probability margin	$\Delta(\mathbf{p}_{\text{RF}}) \geq \tau_{\text{margin}}$	$\tau_{\text{margin}} = 0.20$
In-distribution membership	$D_M^2(\mathbf{x}) \leq \tau_{\text{id}}$	Empirical threshold

3.4.3. Replay buffer and incremental update

Accepted pseudo-labelled pairs $(\mathbf{x}, \hat{y}_{\text{RF}})$ are stored in a first-in, first-out (FIFO) replay buffer with a fixed capacity of 512 entries. The buffer acts as a temporary memory for reliable teacher-generated samples and provides the online LSTM student with a controlled source of incremental updates. At each time step in which the pseudo-label gate accepts a sample, one stored pair is selected uniformly at random from the buffer and used to update the LSTM through a single-sample gradient step. This update is performed only when learning is enabled during the adaptation phase; otherwise, the student parameters remain unchanged (Algorithm 6).

Algorithm 6 Replay-based incremental update.

Require: Incoming sample $\mathbf{x}_t$, RF pseudo-label $\hat{y}_{\text{RF}}$, acceptance indicator $\mathcal{A}(\mathbf{x}_t)$, replay buffer $\mathcal{B}$

```

1: if  $\mathcal{A}(\mathbf{x}_t) = 1$  then
2:   Append  $(\mathbf{x}_t, \hat{y}_{\text{RF}})$  to  $\mathcal{B}$ 
3:   if  $|\mathcal{B}| > 512$  then
4:     Remove the oldest entry from  $\mathcal{B}$ 
5:   end if
6:   Sample  $(\mathbf{x}_r, y_r)$  uniformly at random from  $\mathcal{B}$ 
7:   Update the LSTM student using  $(\mathbf{x}_r, y_r)$ 
8: else
9:   Skip the LSTM update for the current time step
10: end if

```

The replay mechanism improves online learning stability in two main ways. First, it reduces the temporal dependence of consecutive stream samples, preventing the LSTM from being updated only on highly correlated traffic instances. Second, it allows recently accepted samples to be revisited, which helps mitigate catastrophic forgetting when some classes occur infrequently in the current stream window [38,39]. If no pseudo-label satisfies the gate at a given step, no sample is added to the buffer, no replay instance is drawn, and no gradient update is issued. Therefore, the LSTM adapts only from samples that are both confidently labelled by the RF teacher and considered distributionally reliable (Table 7).

Table 7

Replay buffer configuration for online LSTM adaptation.

Parameter	Configuration
Buffer type	First-in, first-out replay memory
Buffer capacity	512 accepted pseudo-labelled samples
Stored item	$(\mathbf{x}, \hat{y}_{\text{RF}})$
Sampling strategy	Uniform random sampling
Update frequency	One update per accepted pseudo-label
Update condition	All pseudo-label gate criteria satisfied
No accepted pseudo-label	No LSTM update issued

3.5. Composite hybrid switching

The proposed framework employs a composite hybrid switching mechanism to determine whether the final prediction at each time step should be produced by the offline RF teacher or the online LSTM student. During the initial warm-up period of the first 100

samples, the RF teacher is used exclusively because the LSTM has not yet received sufficient pseudo-label-based adaptation. After this period, if both models predict the same class, i.e.,

$$\hat{y}_{\text{RF}} = \hat{y}_{\text{LSTM}}. \quad (12)$$

the RF output is selected by default, since model agreement indicates that no source-level conflict needs to be resolved. When the two models disagree, the framework computes a composite confidence score for each model and selects the prediction source according to their relative reliability.

For disagreement cases, the RF confidence score is defined as:

$$s_{\text{RF}} = 0.50 \cdot (1 - \tilde{H}(\mathbf{p}_{\text{RF}})) + 0.30 \cdot \Delta(\mathbf{p}_{\text{RF}}) + 0.20 \cdot \mathbb{1}[D_M^2(\mathbf{x}) \leq \tau_{\text{id}}], \quad (13)$$

where $\tilde{H}(\mathbf{p}_{\text{RF}})$ denotes the normalised entropy of the RF probability vector, $\Delta(\mathbf{p}_{\text{RF}})$ denotes the top-2 probability margin, and $\mathbb{1}[\cdot]$ is the indicator function for in-distribution membership. The in-distribution term is included because the RF teacher is trained and calibrated on the offline training distribution, and its reliability for out-of-distribution samples cannot be assumed.

The LSTM confidence score is computed as:

$$s_{\text{LSTM}} = 0.60 \cdot (1 - \tilde{H}(\mathbf{p}_{\text{LSTM}})) + 0.40 \cdot \Delta(\mathbf{p}_{\text{LSTM}}), \quad (14)$$

where $\tilde{H}(\mathbf{p}_{\text{LSTM}})$ and $\Delta(\mathbf{p}_{\text{LSTM}})$ are defined analogously for the LSTM probability vector. Unlike the RF score, the LSTM score does not include an in-distribution bonus because the student is incrementally adapted from the deployment stream and is not fixed to the original training distribution in the same way.

Algorithm 7 Composite hybrid switching.

Require: RF probabilities $\mathbf{p}_{\text{RF}}$, LSTM probabilities $\mathbf{p}_{\text{LSTM}}$, Mahalanobis distance $D_M^2(\mathbf{x})$, previous source source_{t-1}

Ensure: Final prediction source source_t

```

1: if  $t \leq 100$  then
2:    $\text{source}_t \leftarrow \text{RF}$ 
3: else if  $\hat{y}_{\text{RF}} = \hat{y}_{\text{LSTM}}$  then
4:    $\text{source}_t \leftarrow \text{RF}$ 
5: else
6:   Compute  $s_{\text{RF}}$  using Eq. (13)
7:   Compute  $s_{\text{LSTM}}$  using Eq. (14)
8:   if  $s_{\text{RF}} \geq s_{\text{LSTM}} + \delta_h$  then
9:      $\text{source}_t \leftarrow \text{RF}$ 
10:  else if  $s_{\text{LSTM}} > s_{\text{RF}} + \delta_h$  then
11:     $\text{source}_t \leftarrow \text{LSTM}$ 
12:  else
13:     $\text{source}_t \leftarrow \text{source}_{t-1}$ 
14:  end if
15: end if
16: Store  $\text{source}_t$ , final prediction, probability vector, and uncertainty flag

```

Table 8
Configuration of the composite hybrid switching mechanism.

Component	Configuration
Warm-up period	First 100 samples
Default warm-up source	RF teacher
Agreement rule	Select RF when $\hat{y}_{\text{RF}} = \hat{y}_{\text{LSTM}}$
RF score terms	Entropy, margin, in-distribution membership
LSTM score terms	Entropy, margin
Hysteresis margin	$\delta_h = 0.03$
Initial source	RF teacher
Uncertainty flag	$\min(\tilde{H}_{\text{RF}}, \tilde{H}_{\text{LSTM}}) \geq 0.70$
Logged diagnostics	Source attribution, final prediction, probability vector, uncertainty status

To prevent rapid alternation between the RF and LSTM sources, a hysteresis margin $\delta_h = 0.03$ is applied. The selected source at time step t is defined as:

$$\text{source}_t = \begin{cases} \text{RF}, & \text{if } s_{\text{RF}} \geq s_{\text{LSTM}} + \delta_h, \\ \text{LSTM}, & \text{if } s_{\text{LSTM}} > s_{\text{RF}} + \delta_h, \\ \text{source}_{t-1}, & \text{otherwise.} \end{cases} \quad (15)$$

At initialisation, the default source is set to RF. Thus, when neither model exceeds the other by the hysteresis margin, the previous prediction source is retained. This stabilises the decision process and avoids frequent source switching when the two confidence scores are nearly equal (Algorithm 7).

For each sample, the selected source, final prediction, associated probability vector, and uncertainty status are stored for post-hoc analysis. A sample is flagged as uncertain when both models exhibit high predictive uncertainty, defined as:

$$\min(\tilde{H}_{\text{RF}}, \tilde{H}_{\text{LSTM}}) \geq 0.70. \quad (16)$$

The rolling proportion of RF-sourced and LSTM-sourced predictions is then used as a label-free diagnostic to analyse the behaviour of the hybrid IDS during deployment (Table 8).

4. Experimental setup

4.1. Deployment architecture

The proposed hybrid IDS was evaluated using a two-tier architecture designed to reflect a practical IoT deployment. Computationally intensive tasks, including offline RF training, probability calibration, covariance estimation, and the preparation of deployment artefacts, were performed on the server (Fig. 2). The online LSTM student model was subsequently evaluated under a resource-constrained gateway profile using CPU-only, single-threaded execution. This separation ensured that the gateway experiments primarily represented operational deployment requirements, particularly real-time inference latency, memory consumption, and sequential data processing.

The principal hardware and software configurations employed in the experiments are summarised in Table 9.

Table 9
Experimental hardware and software environment.

Component	Configuration
Operating system	Ubuntu 22.04.5 LTS, 64-bit
Processor	AMD Ryzen Threadripper PRO 5975WX, 32 cores and 64 threads
System memory	251 GiB ECC DDR4
Graphics processor	NVIDIA RTX A5500 with 24 GB GDDR6
Python environment	Python 3.10.12
Main frameworks	PyTorch 2.2.2, deep-river 0.2.6, River 0.21.2, and scikit-learn 1.4.2
Supporting libraries	NumPy 1.26.4, SciPy 1.15.3, Pandas 2.2.3, and psutil 7.2.2

The RF teacher model was trained using all available processor cores to reduce offline training time. During stream-based inference, however, execution was restricted to a single thread because parallel processing may introduce unnecessary overhead when observations are evaluated individually.

The LSTM student received 45 input features and contained approximately 10,310 trainable parameters. Its recurrent representation was passed through a rectified linear unit activation function and a fully connected output layer corresponding to the six target classes. The model parameters required approximately 40.3 KiB when represented using 32-bit floating-point values, excluding framework-related memory overhead.

The teacher model's class probabilities were calibrated using a held-out portion of the offline training data. For drift detection, the mean feature vector and precision matrix were estimated using the Ledoit–Wolf covariance method. These statistics were stored using 32-bit precision to reduce the size of the deployment artefacts and were temporarily converted to 64-bit precision during distance calculations to improve numerical stability.

The IoT gateway was simulated using a dedicated CPU-only execution profile. GPU access, graphical output, and multithreaded numerical processing were disabled. Feature vectors were processed sequentially, thereby avoiding the need to load the complete dataset into memory.

For a stream containing approximately 242,000 observations, the three prediction arrays required approximately 2.77 MiB when stored as 32-bit integers. The 99th-percentile latency was estimated from a rolling window containing the most recent 1000 observations. This procedure limited memory consumption while preserving an indication of upper-tail response time.

A global random seed of 42 was applied to Python, NumPy, and the CPU and GPU random-number generators used by PyTorch. Deterministic GPU behaviour was encouraged by disabling automatic benchmarking and enabling deterministic implementations where supported.

4.2. Dataset and preprocessing

The proposed framework was evaluated using the CICIoMT2024 dataset, a recent benchmark designed to reflect the complexity of Internet of Medical Things (IoMT) environments [40]. The dataset contains traffic generated by both real and simulated IoMT devices and includes heterogeneous communication protocols such as Wi-Fi, MQTT, and Bluetooth. It covers benign traffic and several attack behaviours, including distributed denial-of-service (DDoS), denial-of-service (DoS), reconnaissance, MQTT-based flooding, malformed data, and spoofing attacks, making it suitable for evaluating intrusion detection under realistic and imbalanced IoT network conditions.

The raw dataset was provided as multiple CSV files, each representing either benign traffic or a specific attack type, with separate directories for training and testing. These files were merged into two unified datasets, denoted as *Train.csv* and *Test.csv*. Class labels were derived from file names and then cleaned by removing protocol descriptors, file suffixes, and embedded numeric counters. Inconsistent numeric entries in selected feature columns were converted into missing values, after which rows containing missing entries and duplicate records were removed. This cleaning process reduced the training set from 7,160,831 to 7,155,702 samples and the test set from 1,614,182 to 1,612,117 samples.

To define a compact multi-class IDS task, the original attack labels were consolidated into six target classes: *DDoS*, *DoS*, *MQTT*, *Benign*, *Recon*, and *Spoofing*. DDoS and DoS variants were grouped according to their attack family, MQTT flooding and malformed-data attacks were assigned to the MQTT class, scan-based attacks were grouped as Recon, and ARP spoofing was retained as Spoofing. This mapping reduces label fragmentation while preserving the main behavioural categories required for multi-class intrusion detection.

For the simulation, 50% stratified samples were drawn from both the cleaned training and test files. The training sample formed the offline labelled pool with 3,577,853 samples, which was later divided into an RF-fit set and a calibration set. The test sample formed the deployment pool with 806,059 samples, of which 564,241 were used as a validation stream with LSTM adaptation enabled, and 241,818 were reserved as a held-out test stream with adaptation disabled. This partitioning preserves the separation between offline training, calibration, online adaptation, and final held-out streaming analysis.

The resulting class distributions confirm the highly imbalanced nature of the task, with DDoS and DoS samples dominating the data and minority classes such as Recon and Spoofing appearing far less frequently. Table 10 presents the class distribution after stratified sampling.

Table 10
Class distribution after 50% stratified sampling.

Class	Offline labelled pool	Validation stream	Held-out test stream
DDoS	2,389,594	373,366	160,015
DoS	902,760	145,837	62,501
MQTT	131,469	22,301	9557
Benign	96,366	13,163	5641
Recon	49,640	8964	3842
Spoofing	8024	610	262
Total	3,577,853	564,241	241,818

4.3. Evaluation metrics

The proposed study addresses a supervised multi-class intrusion detection task in which IoT network traffic is classified into benign and attack categories using a hybrid offline–online RF–LSTM framework. The validation strategy follows a streaming evaluation protocol: ground-truth labels are withheld during online inference and adaptation, and all supervised performance metrics are computed only post-hoc after the stream has been processed. This preserves the label-free nature of the deployment loop while still enabling rigorous offline auditing of classification performance, calibration quality, temporal stability, adaptation behaviour, and computational efficiency.

4.3.1. Classification metrics

Classification performance is assessed using accuracy, class-wise precision, recall, F1-score, macro-averaged and weighted-averaged scores, the Matthews correlation coefficient (MCC) [41], and confusion matrices. Let C denote the number of classes, N the total number of evaluated samples, and TP_c , FP_c , FN_c , and TN_c denote the true positives, false positives, false negatives, and true negatives for class c , respectively.

Accuracy measures the proportion of correctly classified samples over the entire evaluation set:

$$\text{Accuracy} = \frac{\sum_{c=1}^C TP_c}{N}. \quad (17)$$

A high accuracy indicates strong overall correctness, whereas a low value suggests frequent misclassification. However, in imbalanced IoT intrusion datasets, accuracy alone may be misleading because dominant classes can obscure poor detection of rare attack types. Therefore, accuracy is reported only as a general summary and is interpreted together with class-balanced metrics.

For each class c , precision, recall, and F1-score are computed as:

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \quad (18)$$

$$\text{Recall}_c = \frac{TP_c}{TP_c + FN_c}, \quad (19)$$

$$\text{F1}_c = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}. \quad (20)$$

Precision indicates the reliability of positive predictions for a given class. A high precision means that when the model assigns traffic to class c , it is usually correct; a low precision indicates many false alarms for that class. Recall, also called sensitivity or detection rate, measures how successfully the model retrieves actual samples of class c . A high recall indicates that few attacks of that class are missed, while a low recall reflects vulnerability to undetected intrusions. The F1-score is the harmonic mean of precision and recall and is especially useful when the two metrics diverge. A high F1-score indicates a strong balance between avoiding false alarms and detecting true attacks.

Macro-averaged metrics are computed by assigning equal importance to every class:

$$\text{Macro-F1} = \frac{1}{C} \sum_{c=1}^C \text{F1}_c. \quad (21)$$

The same averaging rule is applied to macro precision and macro recall. Macro averaging is particularly important in multi-class IDS evaluation because it prevents majority classes from dominating the reported performance. In imbalanced datasets, a lower macro-F1 may reveal weaknesses in minority or low-frequency attack classes even when overall accuracy remains high.

Weighted-averaged metrics incorporate the support of each class. Let $|S_c|$ be the number of true samples belonging to class c . Weighted F1 is defined as:

$$\text{Weighted-F1} = \frac{\sum_{c=1}^C |S_c| \cdot \text{F1}_c}{\sum_{c=1}^C |S_c|}. \quad (22)$$

Weighted averages reflect performance under the actual class distribution of the dataset. A high weighted-F1 indicates strong performance on the traffic composition observed in practice, but it may still underrepresent rare attacks. For this reason, both macro and weighted scores are reported: macro metrics assess fairness across classes, while weighted metrics summarize distribution-aware operational performance.

To provide a robust scalar measure under class imbalance, the Matthews correlation coefficient is also reported. For a multi-class confusion matrix M , MCC can be interpreted as a correlation between the true and predicted labels. Its value ranges from -1 to $+1$, where $+1$ indicates perfect classification, 0 indicates random-level prediction, and -1 indicates complete disagreement. MCC is useful because it accounts for true positives, true negatives, false positives, and false negatives simultaneously, making it more informative than accuracy in imbalanced multi-class intrusion detection.

The confusion matrix provides a detailed error structure that scalar metrics cannot capture. Each entry M_{ij} represents the number of samples whose true class is i and predicted class is j . A row-normalised confusion matrix is also used:

$$\widetilde{M}_{ij} = \frac{M_{ij}}{\sum_{j'=1}^C M_{ij'}}. \quad (23)$$

The diagonal values of the normalised matrix represent class-wise recall, while off-diagonal values reveal systematic confusions between attack categories. This is particularly important for IoT intrusion detection, where attacks such as DoS and DDoS may share similar traffic patterns and may therefore be confused by the classifier.

4.3.2. Ranking and discrimination metrics

When the model produces probability scores rather than only hard labels, ranking-based evaluation is also required. Receiver operating characteristic (ROC) curves and precision–recall (PR) curves are used to assess how well the model separates classes across decision thresholds.

For each class c , a one-vs-rest ROC curve is constructed by treating class c as the positive class and all remaining classes as negative. The true positive rate and false positive rate at threshold τ are:

$$\text{TPR}_c(\tau) = \frac{TP_c(\tau)}{TP_c(\tau) + FN_c(\tau)}, \quad (24)$$

$$\text{FPR}_c(\tau) = \frac{FP_c(\tau)}{FP_c(\tau) + TN_c(\tau)}. \quad (25)$$

The ROC–AUC is then computed as:

$$\text{AUC}_c = \int_0^1 \text{TPR}_c d(\text{FPR}_c). \quad (26)$$

A higher ROC–AUC indicates stronger discrimination between the target class and all other classes across thresholds. A value close to 1 denotes excellent separability, whereas a value near 0.5 indicates random ranking. In the multi-class setting, macro one-vs-rest and one-vs-one AUC scores may be reported to summarize discrimination across all classes.

Precision–recall curves are also reported because they are more informative under class imbalance. At each threshold, precision and recall are computed, and the average precision for class c is defined as:

$$\text{AP}_c = \sum_k (R_k - R_{k-1}) P_k, \quad (27)$$

where P_k and R_k denote precision and recall at the k -th threshold. A high average precision indicates that the model maintains high precision while retrieving a large proportion of the class. In intrusion detection, PR curves are especially useful for evaluating minority attack classes because they focus directly on positive detections rather than being dominated by true negatives.

4.3.3. Intrusion-detection-specific operational rates

In security applications, conventional classification metrics must be complemented by operational rates that reflect the practical consequences of false alarms and missed attacks. The false positive rate, false negative rate, and detection rate are therefore computed for each class and, where appropriate, for a binary benign-versus-attack formulation.

The false positive rate is defined as:

$$\text{FPR}_c = \frac{FP_c}{FP_c + TN_c}. \quad (28)$$

In an IDS, FPR represents the proportion of non- c traffic incorrectly assigned to class c . When the positive class is “Attack”, a high FPR means benign traffic is frequently flagged as malicious, which can overload security analysts, increase alert fatigue, and reduce trust in the system.

The false negative rate is defined as:

$$\text{FNR}_c = \frac{FN_c}{FN_c + TP_c}. \quad (29)$$

FNR measures the proportion of actual class- c samples that the model fails to detect. In security contexts, this metric is critical because false negatives correspond to undetected intrusions. A low FNR is therefore essential for maintaining defensive coverage.

The detection rate is equivalent to recall:

$$\text{DR}_c = \frac{TP_c}{TP_c + FN_c} = 1 - \text{FNR}_c. \quad (30)$$

A high detection rate indicates that the IDS successfully identifies most attacks of the target class. In addition to multi-class detection rates, a binary IDS report is produced by collapsing all attack categories into a single “Attack” class and comparing it against “Benign”. This binary view enables comparison with prior IDS studies that evaluate only anomaly detection or benign-versus-malicious classification.

4.3.4. Probability calibration metrics

Because the hybrid framework uses probability estimates for pseudo-label acceptance, uncertainty monitoring, and switching between model components, calibration quality is evaluated explicitly. A well-calibrated model assigns confidence scores that correspond closely to empirical correctness. For example, among samples predicted with 90% confidence, approximately 90% should be correct.

The multiclass Brier score measures the mean squared distance between predicted probabilities and one-hot encoded true labels:

$$\text{BS} = \frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C (\hat{p}_{ic} - \mathbb{1}[y_i = c])^2, \quad (31)$$

where $\hat{p}_{ic}$ is the predicted probability that sample i belongs to class c , y_i is the true label, and $\mathbb{1}[\cdot]$ is the indicator function. Lower Brier scores indicate more accurate and better-calibrated probability estimates. A score close to zero reflects highly reliable probabilistic prediction, while a larger value indicates poor probability assignment.

Expected calibration error quantifies the discrepancy between confidence and empirical accuracy. Predictions are divided into B confidence bins. Let S_b be the set of samples assigned to bin b , $\text{acc}(S_b)$ the empirical accuracy in that bin, and $\text{conf}(S_b)$ the average predicted confidence. ECE is computed as:

$$\text{ECE} = \sum_{b=1}^B \frac{|S_b|}{N} |\text{acc}(S_b) - \text{conf}(S_b)|. \quad (32)$$

A low ECE indicates that the model’s confidence is aligned with its actual correctness. A high ECE indicates overconfidence or underconfidence. Reliability diagrams are used to visualize this relationship by comparing bin-wise confidence with bin-wise accuracy. In this study, calibration is particularly important because overconfident incorrect predictions may corrupt online adaptation through erroneous pseudo-labels.

4.3.5. Temporal and adaptation metrics

Since the proposed IDS operates on a data stream, static end-of-stream metrics are insufficient. Temporal metrics are used to evaluate how performance evolves under non-stationary traffic and whether online adaptation improves or degrades the model over time.

Rolling macro-F1 is computed over a sliding window of width w :

$$\text{F1}_{\text{roll}}(t) = \text{MacroF1}(\hat{y}_{t-w+1:t}, y_{t-w+1:t}). \quad (33)$$

This metric shows local performance trends throughout the stream. A high and stable rolling F1 indicates consistent detection performance, whereas sharp drops may indicate concept drift, distributional shift, or instability in online adaptation.

Forgetting analysis is used to determine whether adaptation to new samples causes degradation in previously learned knowledge. Around each detected drift or adaptation event e , the pre-event and post-event rolling macro-F1 values are compared:

$$\Delta_e = F_{\text{post}}(e) - F_{\text{pre}}(e). \quad (34)$$

A negative Δ_e indicates performance degradation after adaptation, while a value near zero suggests stability. Recovery time may also be measured as the number of samples required for rolling F1 to return to its pre-event level. This analysis is useful for distinguishing beneficial adaptation from catastrophic forgetting.

Entropy-based temporal diagnostics are also used to evaluate uncertainty over time. For a probability vector p , normalised entropy is computed as:

$$H(p) = -\frac{\sum_{c=1}^C p_c \log p_c}{\log C}. \quad (35)$$

A value close to zero indicates high confidence, while a value close to one indicates maximum uncertainty. Increasing entropy over time may suggest unfamiliar traffic, drift, or reduced model reliability. When combined with rolling performance, entropy helps determine whether uncertainty estimates are consistent with actual errors.

4.3.6. Computational efficiency metrics

Real-time IoT intrusion detection requires not only high predictive performance but also efficient execution. Therefore, the evaluation includes training time, per-sample latency, stage-level latency, and throughput.

Training time is measured as the wall-clock time required to train the offline model and perform calibration. Although this cost is incurred before deployment, it is still reported to characterise the practical cost of model preparation.

Per-sample latency measures the time required to process one incoming sample, including preprocessing, inference, switching, drift monitoring, and online update when applicable. If ℓ_i denotes the latency for sample i , the mean latency is:

$$\bar{\ell} = \frac{1}{N} \sum_{i=1}^N \ell_i. \quad (36)$$

In addition to the mean, the 95th and 99th percentile latencies are reported because real-time systems must control worst-case delay, not only average delay. A low p99 latency indicates that the system remains responsive for nearly all samples.

Throughput is computed as:

$$\text{Throughput} = \frac{N}{\sum_{i=1}^N \ell_i / 1000}, \quad (37)$$

where ℓ_i is measured in milliseconds. Throughput is expressed in samples per second. A high throughput indicates that the IDS can handle high-volume network traffic, while low throughput may limit deployment in busy IoT gateways or monitoring servers.

Stage-level latency decomposition is also reported to identify computational bottlenecks. The total processing time may be divided into feature scaling, drift detection, RF inference, LSTM prediction, hybrid decision selection, and online update. This decomposition supports practical optimisation by showing which component dominates runtime.

5. Results

5.1. Overall classification performance

Table 11 present the classification results of the proposed hybrid RF-LSTM model on the validation and held-out test streams, respectively. Ground-truth labels were used only for post-hoc evaluation and were not accessed during streaming inference or online adaptation.

Table 11
Classification report for the validation and the held-out test stream.

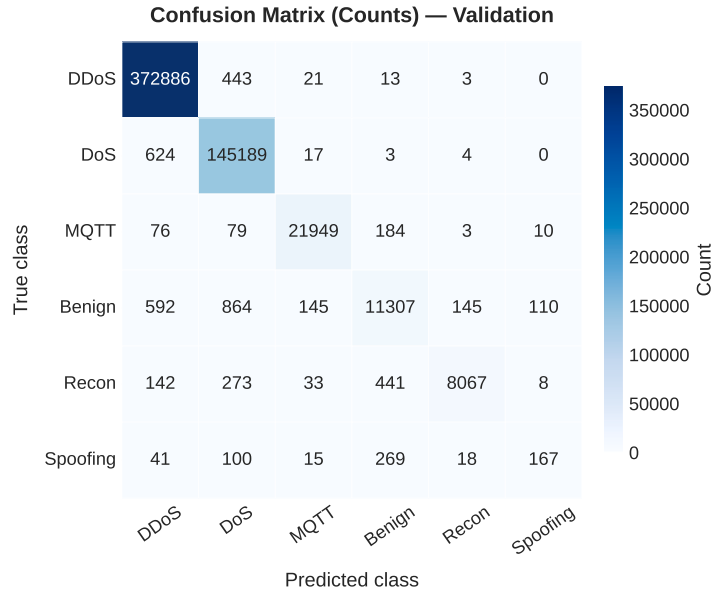
		precision	recall	f1-score	support
Validation	DDoS	0.9961	0.9987	0.9974	373,366
	DoS	0.9880	0.9956	0.9918	145,837
	MQTT	0.9896	0.9842	0.9869	22,301
	Benign	0.9255	0.8590	0.8910	13,163
	Recon	0.9790	0.8999	0.9378	8964
	Spoofing	0.5661	0.2738	0.3691	610
	accuracy			0.9917	564,241
	macro avg	0.9074	0.8352	0.8623	564,241
	weighted avg	0.9913	0.9917	0.9914	564,241
Test (held-out)		precision	recall	f1-score	support
	DDoS	0.9961	0.9977	0.9969	160,015
	DoS	0.9767	0.9946	0.9856	62,501
	MQTT	0.9913	0.9825	0.9869	9557
	Benign	0.9338	0.7876	0.8545	5641
	Recon	0.9811	0.9055	0.9418	3842
	Spoofing	0.8761	0.3779	0.5280	262
	accuracy			0.9893	241,818
	macro avg	0.9592	0.8410	0.8823	241,818
	weighted avg	0.9891	0.9893	0.9889	241,818

The model achieved an accuracy of 0.9917 and a weighted F1-score of 0.9914 on the validation stream. Comparable performance was retained on the held-out test stream, where accuracy and weighted F1 reached 0.9893 and 0.9889, respectively. Test macro F1 increased from 0.8623 to 0.8823, indicating slightly stronger average performance across the six classes.

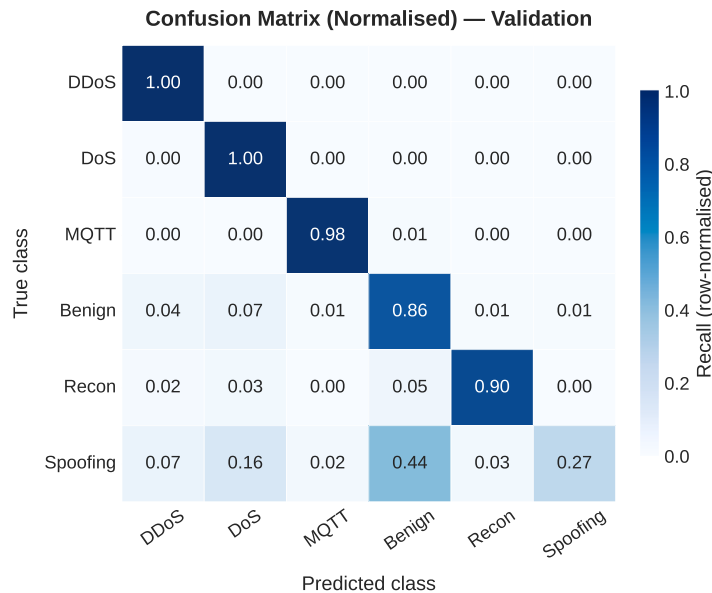
DDoS, DoS, and MQTT achieved F1-scores above 0.98 in both phases, while Recon remained stable at approximately 0.94. Benign performance declined moderately on the test stream, primarily because recall decreased from 0.8590 to 0.7876. Spoofing remained the most difficult class, although its F1-score improved from 0.3691 during validation to 0.5280 during testing. The difference between the weighted and macro-averaged results reflects the lower performance and limited support of the minority classes.

Figs. 3 and 4 show the count-based and row-normalised confusion matrices for the two evaluation phases.

Both confusion matrices show strong diagonal classification for DDoS, DoS, MQTT, and Recon. Most residual errors were concentrated in Benign and Spoofing traffic. Benign samples were primarily misclassified as DoS or DDoS, whereas Spoofing samples were

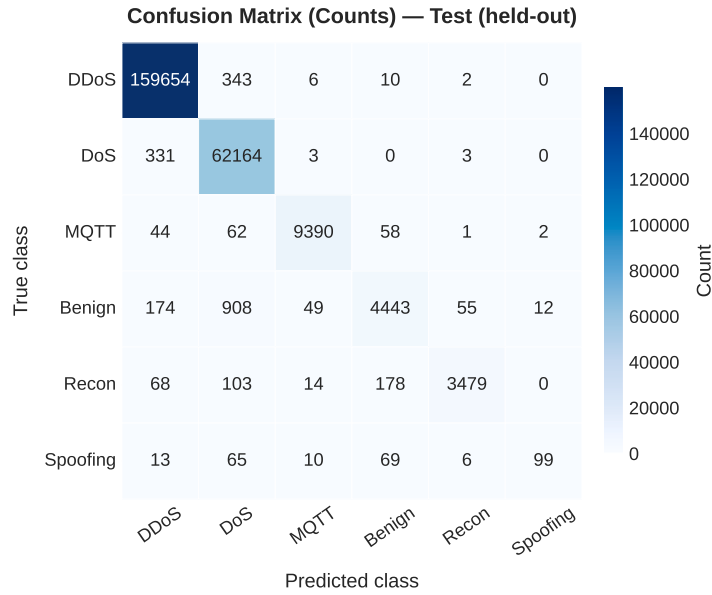


(a) Count-based matrix.

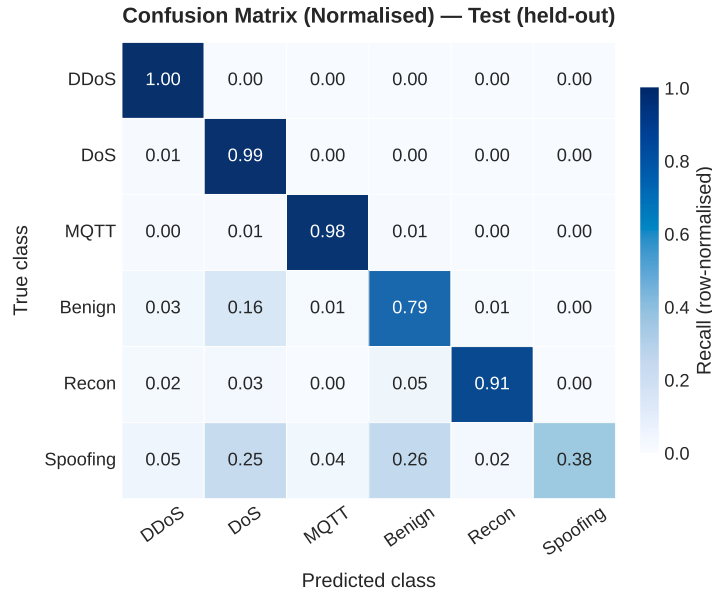


(b) Row-normalised matrix.

Fig. 3. Confusion matrices for the validation stream.



(a) Count-based matrix.



(b) Row-normalised matrix.

Fig. 4. Confusion matrices for the held-out test stream.

most frequently assigned to Benign or DoS. The similar error patterns across validation and testing indicate that the main class-level behaviour was preserved on the held-out stream.

5.2. Ranking and discrimination performance

The ranking performance of the proposed hybrid RF–LSTM model was evaluated using one-vs-rest ROC–AUC and precision–recall average precision (AP). Table 12 summarises the validation and held-out test results.

The model maintained similar macro ROC–AUC values on the validation and held-out test streams, decreasing only from 0.8775 to 0.8727. The corresponding macro AP increased from 0.8432 to 0.8572. DDoS, DoS, and MQTT achieved ROC–AUC and AP values above 0.99 in both phases, while Recon also remained stable, with test ROC–AUC and AP values of 0.9465 and 0.9392.

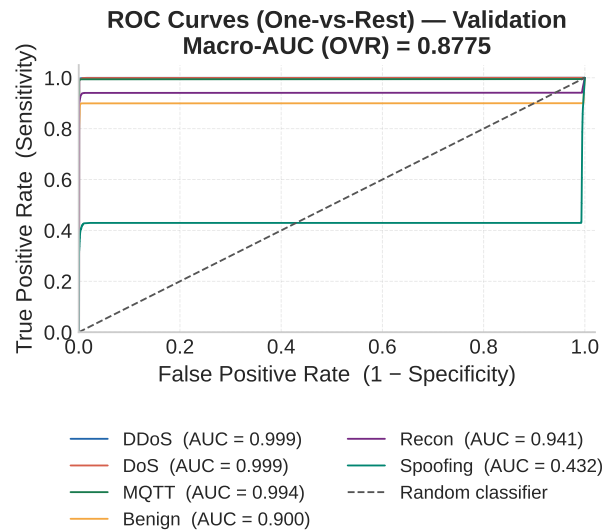
Table 12

ROC-AUC and average precision results on the validation and held-out test streams.

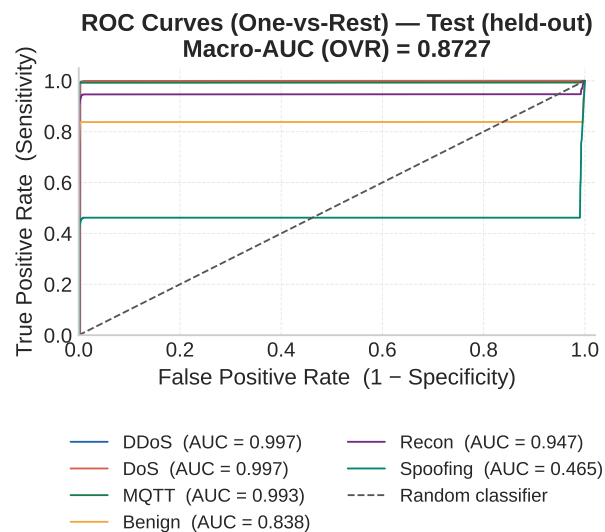
Phase	Metric	DDoS	DoS	MQTT	Benign	Recon	Spoofing	Macro average
Validation	ROC-AUC	0.9992	0.9990	0.9944	0.8996	0.9408	0.4319	0.8775
	AP	0.9996	0.9971	0.9925	0.8802	0.9337	0.2561	0.8432
Test (held-out)	ROC-AUC	0.9967	0.9971	0.9928	0.8380	0.9465	0.4650	0.8727
	AP	0.9982	0.9917	0.9910	0.8136	0.9392	0.4095	0.8572

Benign ranking performance decreased on the test stream, with ROC-AUC declining from 0.8996 to 0.8380 and AP from 0.8802 to 0.8136. Spoofing produced the lowest values, although its ROC-AUC increased from 0.4319 to 0.4650 and its AP from 0.2561 to 0.4095.

Figs. 5 and 6 present the corresponding ROC and precision-recall curves.



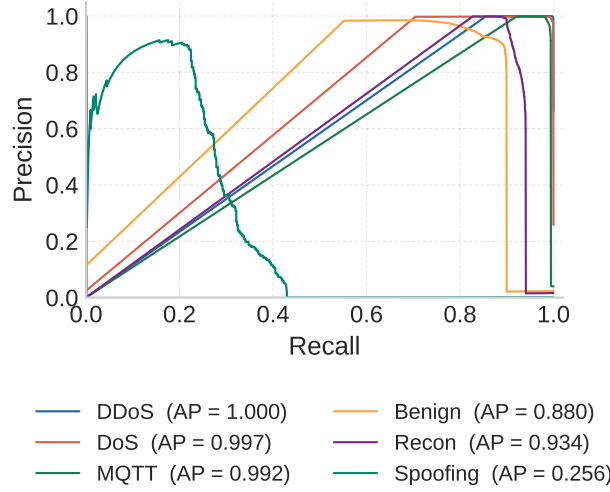
(a) Validation stream.



(b) Held-out test stream.

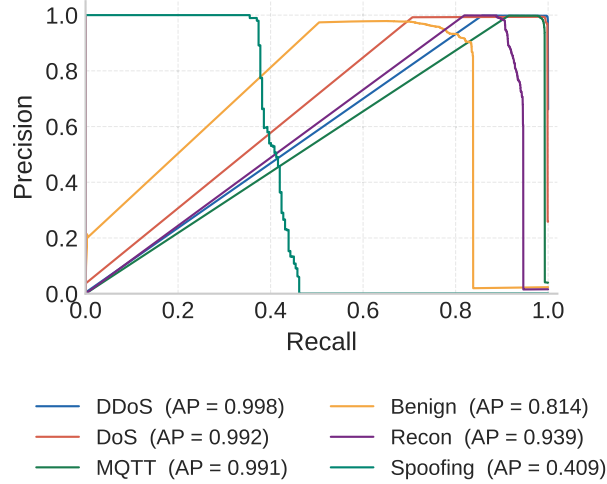
Fig. 5. One-vs-rest ROC curves of the proposed hybrid RF-LSTM model.

Precision–Recall Curves (One-vs-Rest) — Validation
Macro-AP = 0.8432



(a) Validation stream.

Precision–Recall Curves (One-vs-Rest) — Test (held-out)
Macro-AP = 0.8572



(b) Held-out test stream.

Fig. 6. One-vs-rest precision–recall curves of the proposed hybrid RF–LSTM model.

5.3. IDS-specific operational performance

Table 13 presents the class-wise false positive rate (FPR), false negative rate (FNR), and detection rate (DR) for the validation and held-out test streams.

DDoS and DoS achieved detection rates above 0.99 in both phases, while MQTT remained above 0.98. Recon detection increased slightly from 0.8999 to 0.9055. By contrast, Benign detection decreased from 0.8590 to 0.7876. Spoofing remained the most difficult class, although its detection rate improved from 0.2738 to 0.3779 and its FNR decreased from 0.7262 to 0.6221.

For binary IDS evaluation, all attack categories were merged into a single Attack class. The corresponding results are reported in Table 14.

Binary attack recall remained high, increasing from 0.9983 during validation to 0.9987 during testing. Accuracy remained above 0.99 in both phases, reaching 0.9951 and 0.9937, respectively. The main difference was observed for Benign traffic, whose recall decreased from 0.8590 to 0.7876, while the overall attack detection rate remained stable.

Table 13

Class-wise IDS-specific performance on the validation and held-out test streams.

Phase	Class	FPR	FNR	DR	TP	FP	FN
Validation	DDoS	0.0077	0.0013	0.9987	372,886	1475	480
	DoS	0.0042	0.0044	0.9956	145,189	1759	648
	MQTT	0.0004	0.0158	0.9842	21,949	231	352
	Benign	0.0017	0.1410	0.8590	11,307	910	1856
	Recon	0.0003	0.1001	0.8999	8067	173	897
	Spoofing	0.0002	0.7262	0.2738	167	128	443
Test (held-out)	DDoS	0.0077	0.0023	0.9977	159,654	630	361
	DoS	0.0083	0.0054	0.9946	62,164	1481	337
	MQTT	0.0004	0.0175	0.9825	9390	82	167
	Benign	0.0013	0.2124	0.7876	4443	315	1198
	Recon	0.0003	0.0945	0.9055	3479	67	363
	Spoofing	0.0001	0.6221	0.3779	99	14	163

Table 14

Binary Benign-versus-Attack classification performance.

Validation		precision	recall	f1-score	support
	Benign	0.9255	0.8590	0.8910	13,163
	Attack	0.9966	0.9983	0.9975	551,078
	accuracy			0.9951	564,241
	macro average	0.9611	0.9287	0.9443	564,241
	weighted average	0.9950	0.9951	0.9950	564,241
Test (held-out)		precision	recall	f1-score	support
	Benign	0.9338	0.7876	0.8545	5641
	Attack	0.9949	0.9987	0.9968	236,177
	accuracy			0.9937	241,818
	macro average	0.9644	0.8931	0.9257	241,818
	weighted average	0.9935	0.9937	0.9935	241,818

5.4. Probability calibration performance

Table 15 reports the multiclass Brier score and expected calibration error (ECE) computed using 15 confidence bins.

Table 15

Probability calibration results on the validation and held-out test streams.

Phase	Brier Score	ECE
Validation	0.0159	0.0071
Test (held-out)	0.0205	0.0096

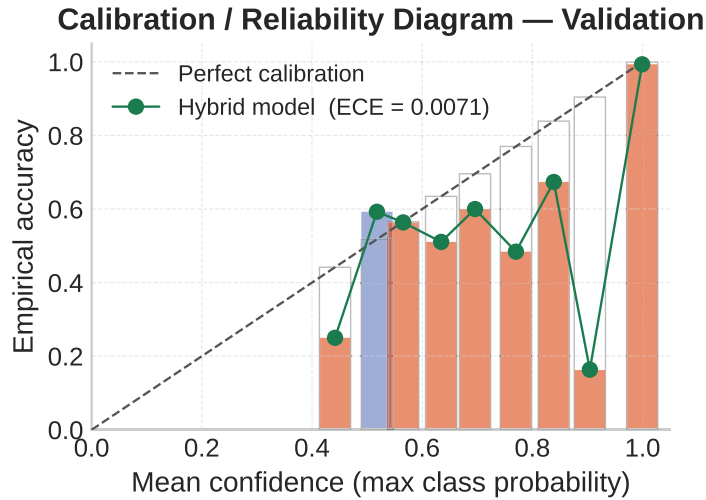
The model remained well calibrated across both evaluation phases. The Brier score increased slightly from 0.0159 on validation to 0.0205 on testing, while ECE increased from 0.0071 to 0.0096. Despite this increase, the low ECE values in both phases indicate close agreement between predicted confidence and empirical accuracy. As shown in Fig. 7, the reliability curves closely follow the perfect-calibration diagonal, indicating that the predicted confidence generally corresponds to the observed classification accuracy. Points below the diagonal represent overconfident predictions, whereas points above it indicate underconfidence. Only small deviations from the diagonal were observed in both streams, although the test curve showed marginally greater variation than the validation curve.

5.5. Temporal performance, forgetting, and predictive uncertainty

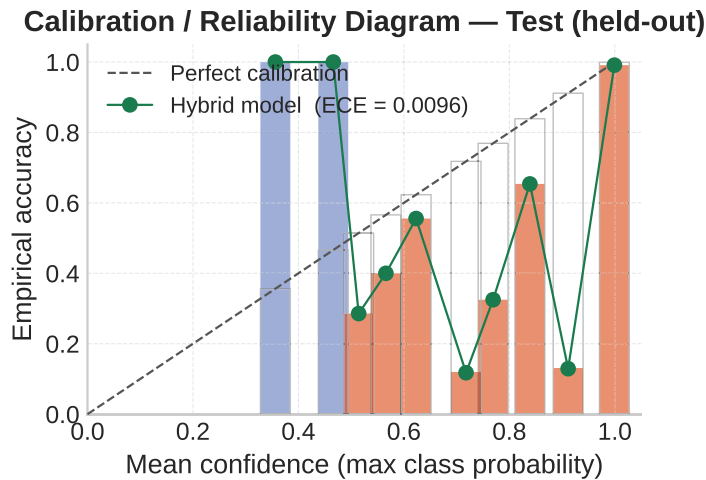
Because the proposed framework operates on ordered data streams, its performance was also examined over time. The analysis considered rolling macro-F1, performance changes around detected drift events, recovery behaviour, and the normalised predictive entropy of the RF teacher and LSTM student. For visual clarity, the figures present representative portions of the complete validation and held-out test streams.

5.5.1. Rolling macro-F1

Fig. 8 presents the post-hoc rolling macro-F1, computed using a window of 200 samples and accompanied by a smoothed trend with a window of 50 samples. Both streams exhibited short-term fluctuations, while the smoothed curves provided a clearer view of the underlying temporal performance pattern.



(a) Validation stream.



(b) Held-out test stream.

Fig. 7. Reliability diagrams of the proposed hybrid RF-LSTM model.

The displayed intervals show that macro-F1 varied across the stream rather than remaining constant. Nevertheless, local decreases were frequently followed by subsequent recovery, and the smoothed trends remained above the lowest short-term values in both evaluation phases. The validation and test plots use the same rolling and smoothing windows, allowing direct comparison of their temporal behaviour.

5.5.2. Forgetting and recovery at drift events

Catastrophic forgetting was assessed using the LSTM-only predictions around Page-Hinkley drift alarms. For each evaluable event, the rolling macro-F1 before and after the alarm was compared using

$$\Delta F1 = F1_{\text{post}} - F1_{\text{pre}}. \quad (38)$$

Events with $\Delta F1 < -0.10$ were classified as severe forgetting, while values between -0.10 and -0.02 were classified as mild forgetting. Non-negative or limited negative changes were treated as no forgetting. The full analysis contained 5612 detected drift events for validation and 2,456 for the held-out test stream.

Fig. 9 shows reduced intervals of the forgetting analysis. The LSTM-only rolling macro-F1 is compared with the hybrid rolling macro-F1 reference, while the shaded regions represent the pre- and post-event measurement windows.

The selected intervals contain both positive and negative post-drift changes. Some alarms were followed by immediate improvements in LSTM macro-F1, whereas others produced mild or severe declines. The hybrid reference generally remained above the LSTM-only curve over the displayed intervals, although both curves showed local temporal variation.

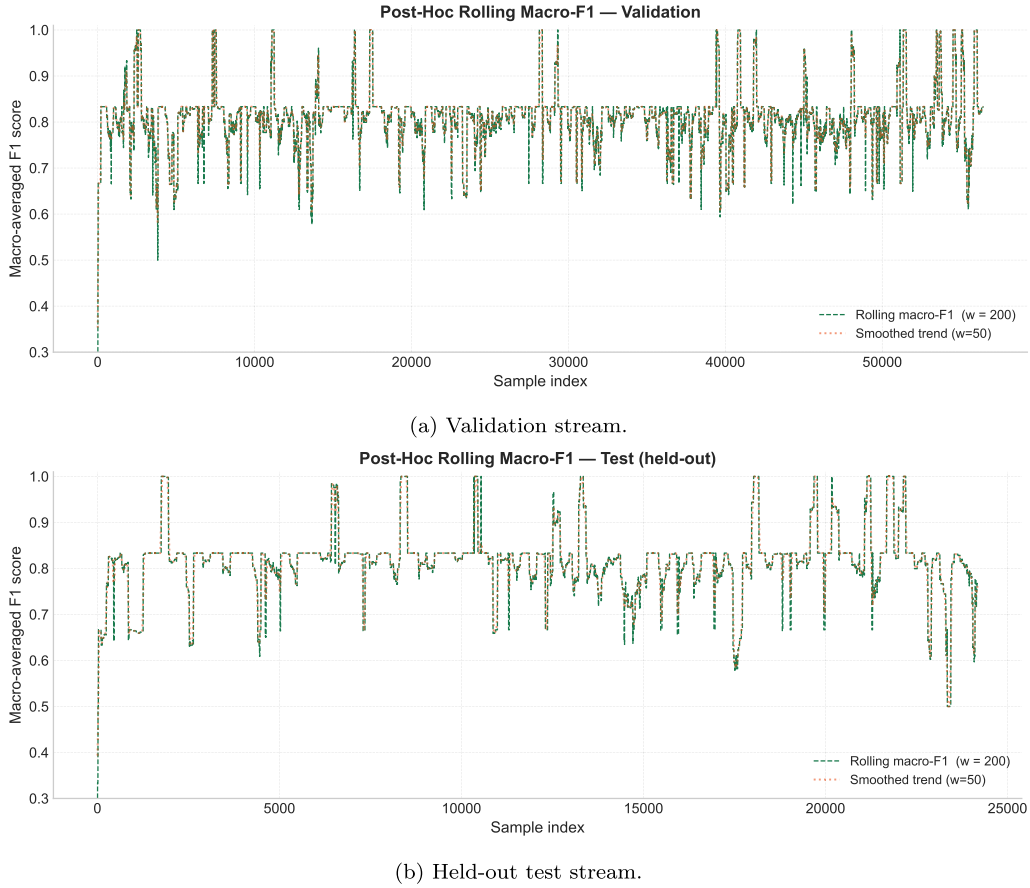


Fig. 8. Post-hoc rolling macro-F1 of the proposed hybrid RF-LSTM model. Reduced stream intervals are shown for clarity.

Fig. 10 provides a direct comparison of the LSTM and hybrid $\Delta F1$ values at a reduced set of drift events. Values below the mild and severe thresholds indicate the corresponding levels of post-event performance degradation.

Across the selected events, both positive and negative $\Delta F1$ values were observed. The event-level variation confirms that drift alarms were not uniformly associated with performance loss; some events were followed by recovery or improvement, while others produced temporary or more pronounced degradation.

5.5.3. Predictive uncertainty

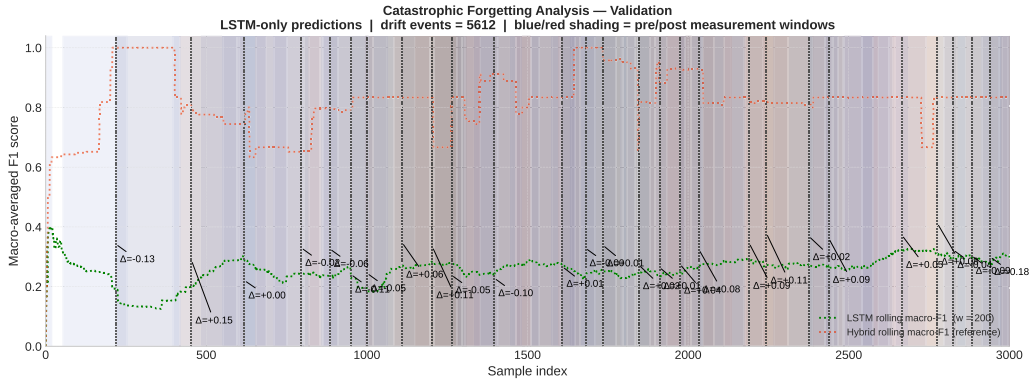
Predictive uncertainty was monitored using normalised Shannon entropy for the RF teacher and LSTM student. Fig. 11 shows the initial stream intervals, while Fig. 12 presents longer representative intervals. The horizontal lines indicate the pseudo-label gate threshold, $H = 0.35$, and the high-uncertainty threshold, $H = 0.70$.

The entropy curves show that uncertainty varied over time for both model components. Low-entropy intervals indicate confident predictions, whereas values above the gate threshold identify samples that would not satisfy the entropy criterion for pseudo-label acceptance. Occasional values approaching or exceeding the high-uncertainty threshold mark periods of substantially reduced predictive confidence. Similar qualitative patterns were observed in the validation and held-out test streams.

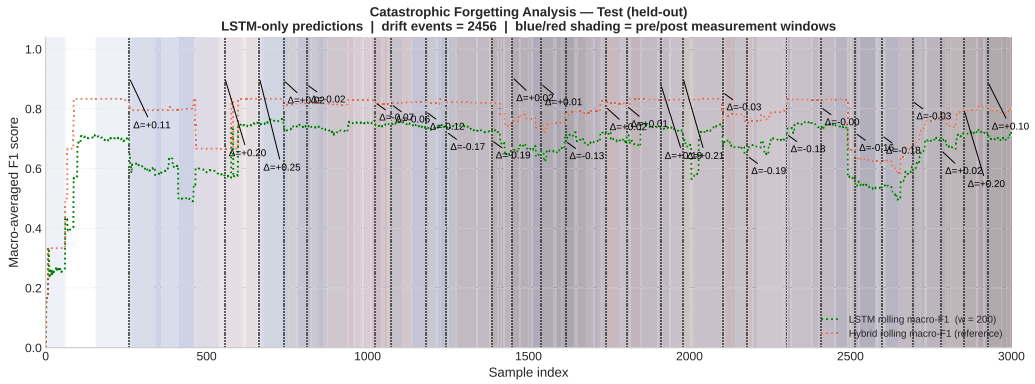
5.6. Computational efficiency

The computational efficiency of the proposed framework was assessed using offline training time, end-to-end latency, throughput, stage-level processing time, and memory usage. The base RF model required 81.59 s for offline training, while the online LSTM performed 552,166 updates during the validation stream.

As shown in Table 16, mean end-to-end latency was 11.99 ms per sample during validation and 12.97 ms during testing. The corresponding p99 latencies were 14.73 and 14.37 ms, indicating that 99% of samples were processed within approximately 15 ms in both phases. Throughput reached 83.42 samples/s on validation and 77.13 samples/s on the held-out test stream.



(a) Validation stream.



(b) Held-out test stream.

Fig. 9. Representative catastrophic-forgetting analysis based on LSTM-only predictions. Reduced event intervals are shown for clarity.**Table 16**

End-to-end computational performance on the validation and held-out test streams.

Phase	Mean (ms)	p95 (ms)	p99 (ms)	Throughput	Peak memory (MB)
Validation	11.99	13.00	14.73	83.42	733.64
Test (held-out)	12.97	13.48	14.37	77.13	325.60

Table 17 reports the stage-level latency profile. RF inference represented the dominant processing cost, with mean latencies of 9.38 ms during validation and 11.51 ms during testing. LSTM prediction required less than 1 ms per sample in both phases. During validation, the active LSTM update added an average of 1.37 ms, whereas its test latency was negligible.

Table 17

Stage-level latency profile of the hybrid RF–LSTM framework.

Phase	Stage	Mean (ms)	p95 (ms)	p99 (ms)
Validation	Feature scaling	0.1638	0.1771	0.1997
	Drift detection	0.0322	0.0379	0.0434
	RF inference	9.3765	10.1625	11.4697
	LSTM prediction	0.7482	0.8081	0.9269
	Pseudo-label gate	0.0947	0.1031	0.1212
	LSTM update	1.3694	1.5089	1.7516
Test (held-out)	Feature scaling	0.1850	0.1925	0.2340
	Drift detection	0.0343	0.0385	0.0477
	RF inference	11.5092	11.9658	12.7282
	LSTM prediction	0.8644	0.8981	1.1596
	Pseudo-label gate	0.1181	0.1255	0.1372
	LSTM update	0.0005	0.0005	0.0008



Fig. 10. LSTM and hybrid macro-F1 changes at selected drift events. The dashed lines denote the mild and severe forgetting thresholds.

Figs. 13, 14, and 15 visualise the stage-level costs, end-to-end latency distributions, and throughput comparison. The latency distribution remained concentrated near the reported central values, although isolated maximum latencies of 1290.32 ms and 598.24 ms were recorded for validation and testing, respectively.

Peak Python-heap allocation measured by `tracemalloc` was 733.64 MB during validation and 325.60 MB during testing. The corresponding RSS changes were 1742.21 and 0.17 MB; however, these values are reported only as contextual measurements because RSS differences are affected by operating-system memory allocation and page reclamation.

6. Discussion

6.1. Main findings and hybrid model trade-Offs

The proposed RF–LSTM framework is best interpreted as an adaptive extension of a strong offline classifier rather than as a replacement intended to maximise static predictive performance. As illustrated in Fig. 16, RF-only achieved the strongest standalone results, whereas LSTM-only was consistently weaker. The hybrid remained closer to the RF baseline, indicating that entropy-based model selection reduced the effect of instability in the online component.

The main benefit of the architecture is therefore its balance between stability and adaptability. The RF provides robust decision boundaries and reliable pseudo-labels, while the LSTM offers an incremental learning path for evolving traffic. Selective updating based on confidence, entropy, margin, and in-distribution criteria enables adaptation without relying unconditionally on the online model.

The hybrid did not surpass RF-only on conventional static metrics; therefore, its contribution should not be framed as universal predictive superiority. Instead, the moderate performance trade-off reflects the addition of label-free adaptation, drift monitoring, and dynamic model selection. The lower RF–LSTM agreement on held-out traffic further suggests that the two components responded differently to unseen conditions, supporting their complementary roles in the streaming IDS.

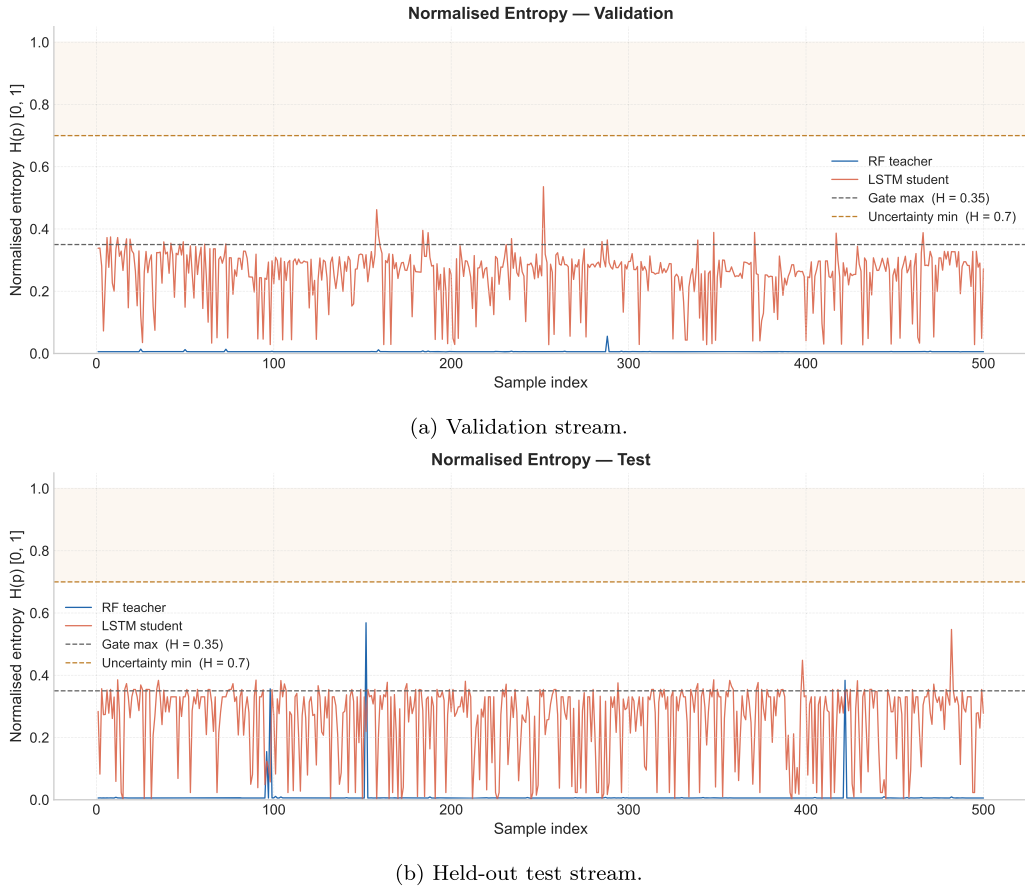


Fig. 11. Normalised entropy during the initial stream intervals.

6.2. Class imbalance, misclassification, and prediction reliability

Class imbalance remained the main source of uneven performance. Although the predicted class proportions closely followed the true distribution, this global agreement concealed systematic under-prediction of minority classes, particularly Spoofing, Recon, and Benign. The difference between weighted and macro metrics therefore reflects strong detection of dominant DDoS, DoS, and MQTT traffic alongside weaker class-balanced generalisation.

The errors were concentrated among related traffic categories rather than being uniformly distributed. Benign samples were most frequently classified as DoS or DDoS, while Spoofing and Recon were often assigned to more prevalent classes. These patterns suggest limited separation in overlapping regions of the feature space, especially for classes with limited training support. As shown in Fig. 17a, close aggregate distributional similarity does not necessarily imply equally reliable detection across all classes.

Prediction reliability also requires cautious interpretation. Despite favourable overall calibration, incorrect predictions were often associated with high confidence, and their confidence distribution overlapped with that of correct predictions (Fig. 17b). This indicates that good global calibration does not eliminate local overconfidence. Class-specific error analysis and uncertainty monitoring are therefore particularly important for rare attacks whose failures may be masked by strong aggregate performance.

6.3. Concept drift, adaptation, and uncertainty

Concept drift did not affect predictive performance uniformly. Although the Page-Hinkley detector generated frequent alarms (Fig. 18a), the rolling-F1 and forgetting analyses showed that only some events were followed by substantial degradation. Other alarms produced limited change or were followed by rapid recovery, indicating that statistical distribution shifts should not automatically be interpreted as harmful performance drift. The feature-level analysis further showed that several early alarms involved simultaneous changes in covariance-, dispersion-, and magnitude-related features, suggesting multidimensional rather than isolated variation (Fig. 18b).

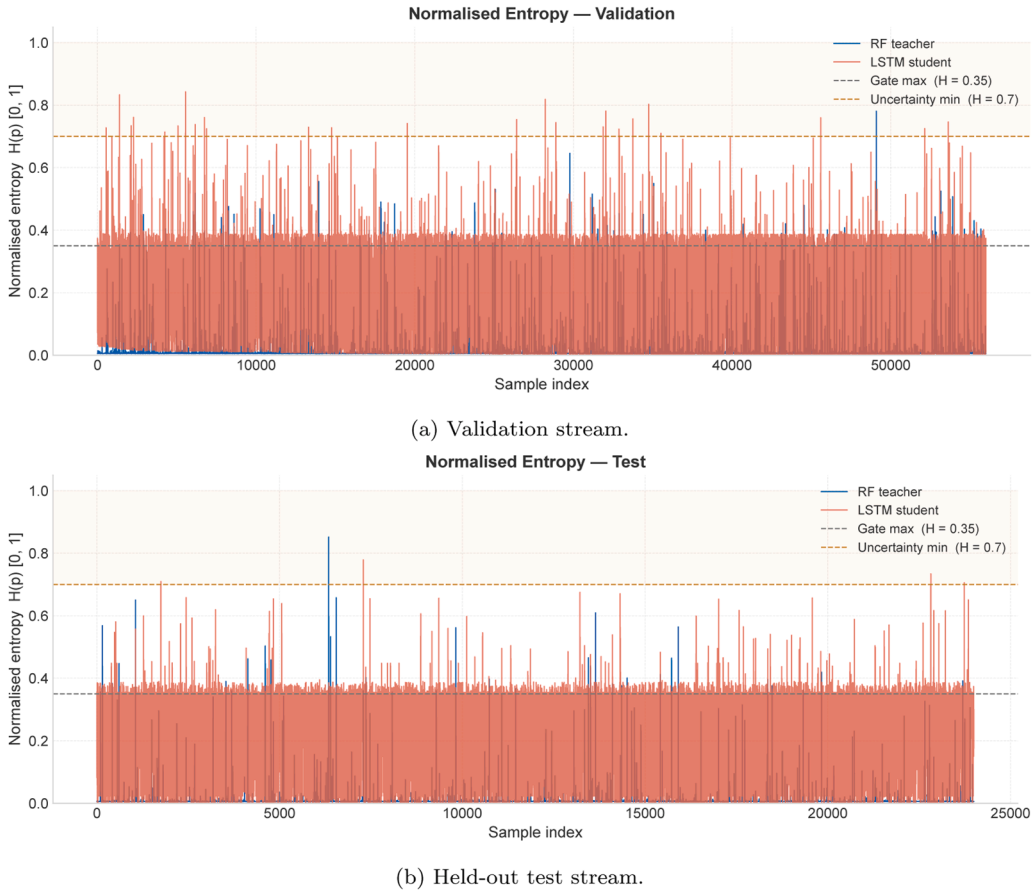


Fig. 12. Normalised entropy over extended representative stream intervals.

Stage-Level Latency Profile

Pseudo-label gate and LSTM update are now timed separately | bars = mean, caps = p99

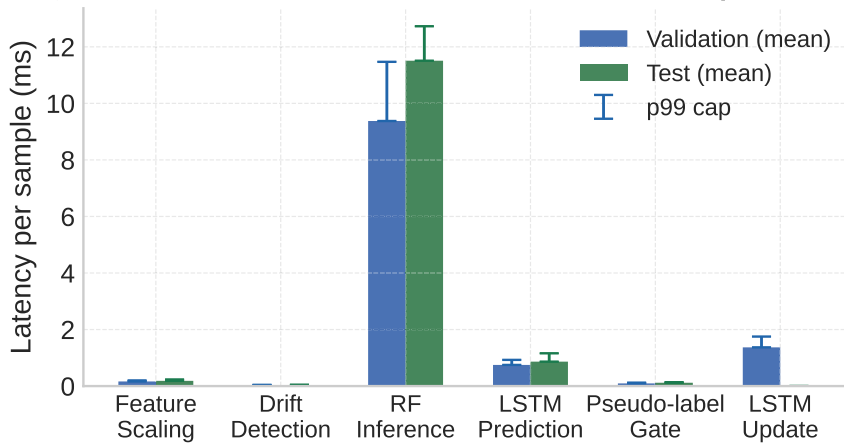


Fig. 13. Stage-level mean and p99 latency of the proposed framework.

The hybrid architecture helped moderate instability in the online component. The LSTM exhibited higher entropy and greater local performance variation than the RF, whereas the hybrid trajectory was generally more stable around detected drift events. This suggests that entropy-based model selection reduced unconditional reliance on the adaptive learner during uncertain periods. The lower RF–LSTM agreement on held-out traffic also indicates that the two components responded differently to unseen conditions, supporting their complementary roles.

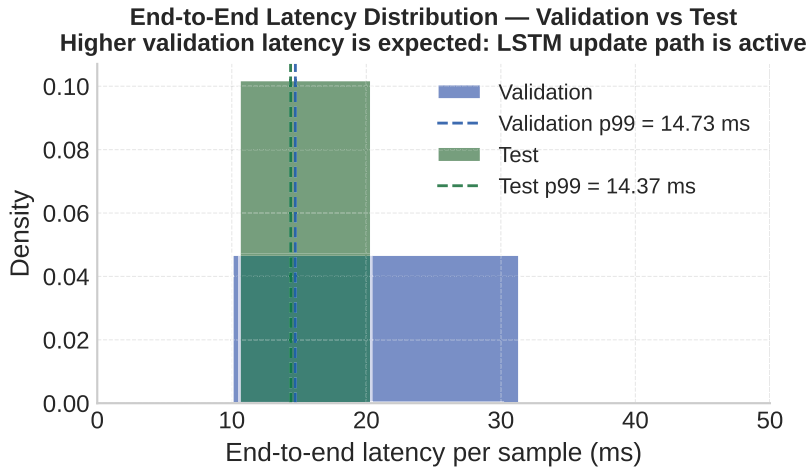


Fig. 14. End-to-end latency distributions for the validation and held-out test streams.

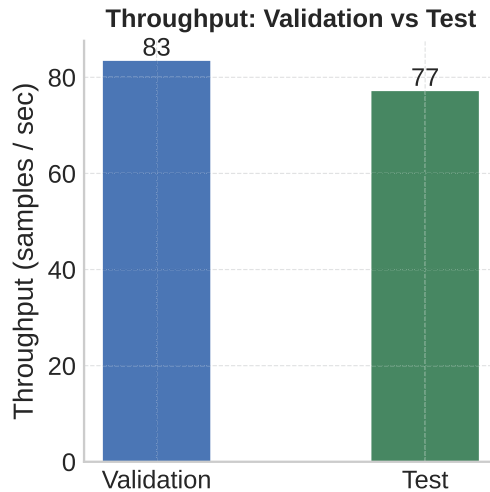


Fig. 15. Throughput comparison between the validation and held-out test streams.

The high pseudo-label acceptance rate indicates that most streamed samples satisfied the adaptation criteria, but it does not guarantee that all accepted labels were correct. Because some misclassifications remained highly confident, confidence, entropy, model agreement, and in-distribution checks should be interpreted jointly. Overall, the findings support controlled online adaptation while highlighting the need to relate drift alarms to performance impact and to limit pseudo-label error propagation and catastrophic forgetting.

6.4. Operational and deployment implications

The operational results suggest that the framework can support real-time intrusion detection under the evaluated workload, although its suitability depends on the target traffic rate and hardware platform. End-to-end latency remained stable at the upper percentiles, while stage-level profiling showed that RF inference dominated the processing cost. Consequently, further runtime optimisation should focus primarily on the offline component rather than on the LSTM update stage.

The IDS-specific findings also reveal a trade-off between attack sensitivity and false-alert burden. The framework maintained a very low missed-attack rate, but the lower recall for Benign traffic on the held-out stream increased the risk of legitimate activity being classified as malicious. Moreover, the F1-optimal decision threshold differed between validation and testing, indicating that a fixed operating point may not generalise across traffic conditions. Deployment thresholds should therefore reflect the relative costs of false alarms and missed attacks.

Memory demand further constrains deployment. The higher validation-phase footprint is consistent with active adaptation and online updating, whereas the test phase operated without learning. Since Python-level peak allocation and RSS change capture different aspects of memory use, they should not be interpreted interchangeably. Overall, the current implementation is more appropriate for an IoT gateway, edge server, or monitoring node than for highly constrained IoT endpoints.

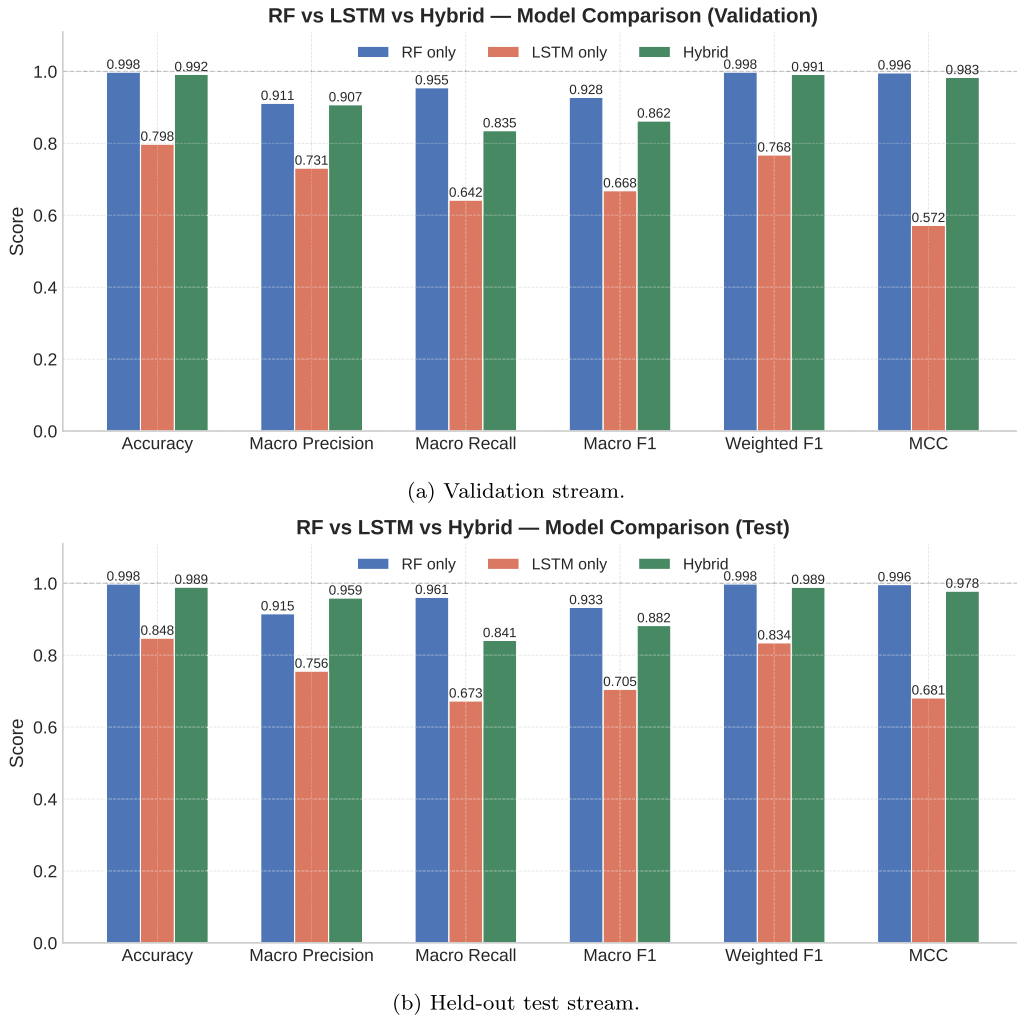


Fig. 16. Comparison of the RF-only, LSTM-only, and hybrid models. The RF provides the strongest static performance, while the hybrid preserves much of this predictive strength while incorporating label-free adaptation and dynamic model selection.

6.5. Comparison with existing studies

The proposed RF–LSTM framework was compared with representative offline, incremental, online, and hybrid intrusion-detection approaches. Since the reported studies differ in dataset composition, task formulation, class balance, metric aggregation, hardware platform, and evaluation protocol, the comparison is intended to position the proposed method rather than establish a strict numerical ranking.

Table 18 provides the most direct comparison because all methods were evaluated on CICIoMT2024. Binary formulations generally report stronger results than multiclass settings, reflecting the lower complexity of separating Benign and Attack traffic compared with distinguishing individual attack families. For consistency, the proposed framework reports macro-averaged precision, recall, and F1-score in both Binary and Multiclass settings. This prevents the dominant Attack class from inflating the binary results and gives equal importance to all classes in the multiclass evaluation. In the multiclass setting, the proposed model exceeds the RF results reported by Dadkhah et al., although it remains below the MultiD-CNN results of Hossain et al. on conventional classification metrics. Unlike these offline approaches, however, the proposed framework also supports label-free adaptation, unsupervised drift monitoring, and held-out streaming evaluation.

The broader comparison in Table 19 further shows that several adaptive and hybrid approaches report higher aggregate scores. These values are not strictly equivalent because the studies use different datasets, numbers of classes, learning modes, and metric-averaging procedures. The proposed framework is therefore reported under both Binary and Multiclass formulations using macro averages. Its lower multiclass macro recall and F1-score mainly reflect the difficulty of distinguishing rare and overlapping attack categories rather than an inability to separate malicious from legitimate traffic. The comparison consequently supports a deployment-oriented interpretation of the framework rather than a claim of universal superiority in static classification.

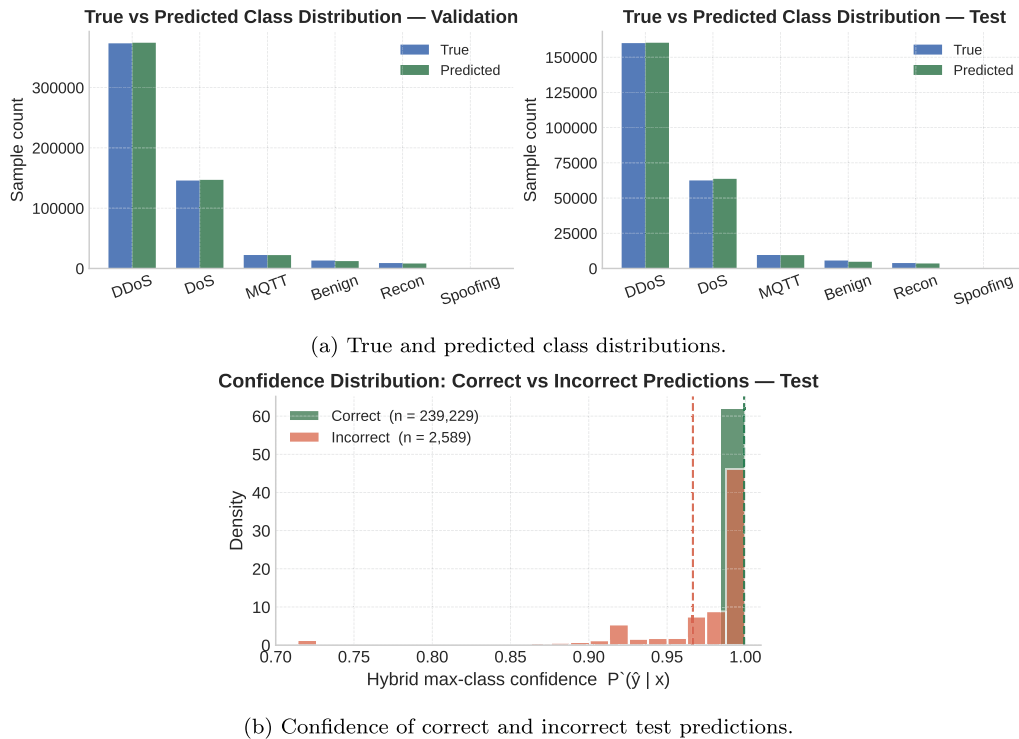


Fig. 17. Class-distribution consistency and prediction-confidence behaviour. Aggregate distributional agreement remains strong, but minority classes are under-predicted and some incorrect decisions retain high confidence.

Table 18

Comparison with existing intrusion-detection methods evaluated on the CICIoMT2024 dataset.

Study	Learning mode	Model	Dataset	Task	Accuracy	Precision	Recall	F1-score
Dadkhah et al. [40]	Offline	RF	CICIoMT2024	Binary	0.9960	0.9710	0.9510	0.9610
				Multiclass	0.7350	0.7350	0.7130	0.6760
Hossain et al. [42]	Offline	MultiD-CNN	CICIoMT2024	Multiclass	0.9811	0.9773	0.9821	0.9788
Lucia et al. [43]	Offline	AdaBoost	CICIoMT2024	Binary	0.9837	0.9917	0.9824	0.9870
Hamad et al. [44]	Offline	TNN-DCNNs-LSTM	CICIoMT2024	Binary	0.9999	1.0000	1.0000	1.0000
Gomez et al. [45]	Offline	CBLOF	CICIoMT2024	Binary	0.9300	0.9000	0.9500	0.9600
Proposed RF-LSTM	Offline teacher + online student	RF-LSTM	CICIoMT2024	Binary	0.9937	0.9644	0.8931	0.9257
				Multiclass	0.9893	0.9592	0.8410	0.8823

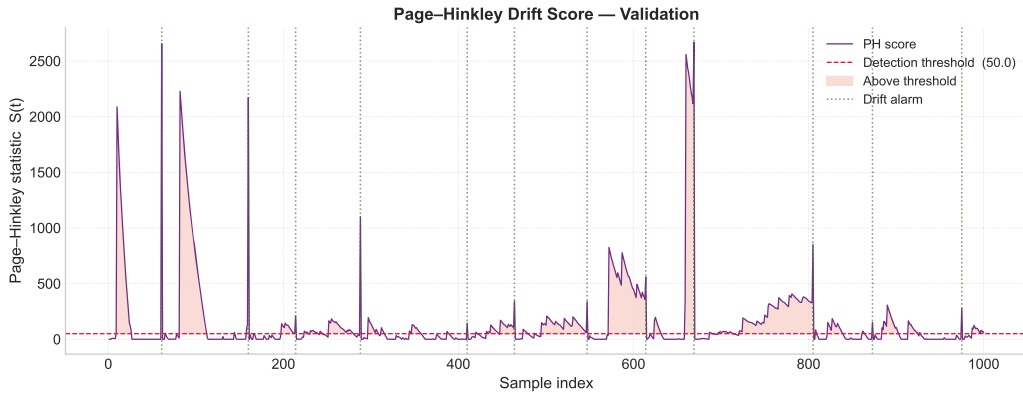
Table 19

Comparison of the predictive performance of adaptive and hybrid intrusion-detection approaches.

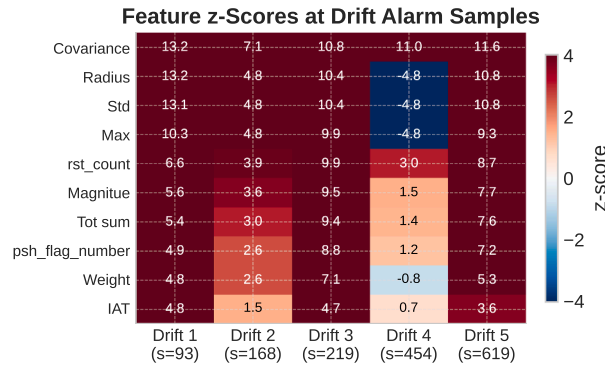
Method	Dataset	Learning mode	Task	Accuracy	Precision	Recall	F1-score
BFS-NID [23]	CIC-IDS2017	Few-shot incremental	Multiclass	0.9317	0.9313	0.9317	0.9295
LightGBM-PA [19]	CICIoT2023	Batch + incremental	Multiclass	0.9935	0.9932	0.9935	0.9932
ALMANET [17]	NF UNSW-NB15	Fully online	Binary	0.9911	–	0.9800	–
QUIC-AID [28]	QUIC dataset	Online + ADWIN	Binary	0.9992	0.9991	0.9994	0.9992
HSF-RF [29]	IoTID20	Offline training	Multiclass	0.9993	–	–	0.9992
Proposed RF-LSTM	CICIoMT2024, held-out test	Offline teacher + online student	Binary	0.9937	0.9644	0.8931	0.9257
			Multiclass	0.9893	0.9592	0.8410	0.8823

Table 20 highlights the operational trade-off between attack sensitivity and false-alert control. The proposed framework achieves very high attack recall and therefore misses relatively few malicious samples, but its weaker recognition of Benign traffic produces a comparatively high false-positive rate. This operating point favours threat coverage over alert reduction and may be appropriate where missed intrusions carry a high cost. However, it may also increase analyst workload and alert fatigue.

The adaptation and efficiency comparison in Table 21 shows that the proposed framework occupies an intermediate position between continuously updated online models and static offline detectors. The LSTM is updated only for samples that satisfy the pseudo-label acceptance criteria, reducing exposure to uncertain updates while preserving an adaptive learning pathway. Its measured latency lies within the broad range reported for several competing approaches, although direct comparison is limited by differences



(a) Page-Hinkley drift behaviour in the validation stream.



(b) Feature deviations at selected drift alarms.

Fig. 18. Statistical drift detection and feature-level changes. Frequent alarms indicate substantial stream variation, while the feature z-scores illustrate that selected events involved simultaneous shifts across multiple traffic characteristics.

Table 20

Comparison of detection reliability, strengths, and limitations.

Method	Reported FPR	Detection rate / attack recall	Main strength	Main limitation
BFS-NID [23]	Up to 0.15	0.95–0.98	Learns new classes from five samples	Confusion among similar attacks
LightGBM-PA [19]	–	0.9935	Strong batch-incremental balance	Weak rare-class AUC
ALMANET [17]	Near zero in rolling evaluation	0.9314–0.9956	Fully online and lightweight	Binary classification only
QUIC-AID [28]	<0.001	0.9994	Strong drift-aware detection	Evaluated on only two attack types
HSF-RF [29]	0.000–0.0133	Approximately 0.998	High accuracy and interpretable features	No continuous adaptation
Proposed RF-LSTM	0.2124	0.9987	High attack sensitivity, label-free adaptation, and good calibration	Elevated false-alert rate and weak minority-class recall

Table 21

Comparison of adaptation mechanisms and computational efficiency.

Method	Adaptation mechanism	Model updating	Drift handling	Latency	Memory
BFS-NID [23]	Session-specific re-projection	Per incremental session	No explicit drift detector	~2.08 ms/sample	2,998.84 MB
LightGBM-PA [19]	Incremental PA updates	Per mini-batch	Dynamic regularisation	20–110 ms	0.34–3.01 MB
ALMANET [17]	ALMA-SWA + online neural network	Per sample	Continuous adaptation	<50 ms/sample	14.64 KB model
QUIC-AID [28]	ARF tree replacement	Per sample	ADWIN	4.69 ms/flow	14.34 MB
HSF-RF [29]	Static RF inference	No online update	None	~1–13 ms/packet	300–357 MB
Proposed RF-LSTM	Confidence-gated LSTM adaptation	Per accepted sample during adaptation	Mahalanobis distance + Page-Hinkley	12.97 ms/sample	325.60 MB peak allocation

in hardware, processing units, and measurement procedures. The memory requirement is higher than that of lightweight online methods, while stage-level profiling identifies RF inference as the main computational bottleneck. The current implementation is therefore more suitable for gateway- or edge-level monitoring than for highly constrained IoT endpoints.

Overall, the proposed RF–LSTM framework does not dominate every reference method in static accuracy, false-positive control, or computational cost. Its distinctive contribution lies in integrating a strong offline teacher, a confidence-gated online learner, entropy-based model selection, unsupervised drift monitoring, and leakage-resistant held-out streaming evaluation within a single IDS.

7. Conclusion

This study proposed a label-free hybrid intrusion detection framework for streaming IoT traffic by integrating an offline RF teacher with an online LSTM student. The framework combines unsupervised drift monitoring, entropy-based model selection, and confidence-gated pseudo-label adaptation within a leakage-resistant streaming evaluation protocol in which ground-truth labels are used only for post-hoc assessment.

The results demonstrate that the proposed approach maintains strong detection performance, reliable probability estimates, and stable behaviour across validation and held-out streams. The RF provides robust baseline predictions, while the LSTM introduces an adaptive learning pathway for evolving traffic. The hybrid mechanism preserves much of the RF's predictive strength while supporting label-free adaptation and uncertainty-aware model selection. The framework also achieves high binary attack detection and consistent multiclass performance, with particularly strong recognition of DDoS, DoS, MQTT, and Recon traffic.

Temporal analysis further shows that the framework can respond to distributional changes while limiting instability in the online component. The measured latency and throughput also support real-time deployment on IoT gateways, edge servers, and dedicated monitoring nodes.

Overall, the proposed RF–LSTM framework provides a practical balance between predictive stability, online adaptability, concept-drift awareness, and deployment-oriented efficiency. Future work will investigate model compression and validation in live IoT environments.

CRediT authorship contribution statement

Dusmurod Kilichev: Writing – original draft, Visualization, Software, Methodology; **Wooseong Kim:** Writing – review & editing, Validation, Supervision, Resources, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Code is available at <https://github.com/TATU-hacker/Real-Time-Intrusion-Detection-in-IoT-Networks>. The CICIoMT2024 dataset is publicly available from the Canadian Institute for Cybersecurity.

References

- [1] A. Alotaibi, A. Barnawi, Securing massive IoT in 6G: recent solutions, architectures, future directions, *Internet Things* 22 (2023) 100715. <https://www.sciencedirect.com/science/article/pii/S2542660523000380>. <https://doi.org/https://doi.org/10.1016/j.iot.2023.100715>
- [2] Current IoT forecast highlights, 2024. Accessed: 2024-12-12 <https://transforminsights.com/research/forecast/highlights>.
- [3] A. Chatterjee, B.S. Ahmed, IoT anomaly detection methods and applications: a survey, *Internet Things* 19 (2022) 100568. <https://www.sciencedirect.com/science/article/pii/S2542660522000622>. <https://doi.org/https://doi.org/10.1016/j.iot.2022.100568>
- [4] V. Merlino, D. Allegra, Energy-based approach for attack detection in IoT devices: a survey, *Internet Things* 27 (2024) 101306. <https://www.sciencedirect.com/science/article/pii/S2542660524002476>. <https://doi.org/https://doi.org/10.1016/j.iot.2024.101306>
- [5] O. Briliyant, A. Javed, Y. Cherdantseva, Beyond the automation gap: a survey of continuous audit for IoT security, *Comput. Secur.* 168 (2026) 104953. <https://www.sciencedirect.com/science/article/pii/S016740482600129X>. <https://doi.org/https://doi.org/10.1016/j.cose.2026.104953>
- [6] M. Yacoubi, O. Moussaoui, C. Drocourt, AI for IoMT security: a comprehensive survey of intrusion detection and system architectures, *Internet Things* 36 (2026) 101869. <https://www.sciencedirect.com/science/article/pii/S254266052500383X>. <https://doi.org/https://doi.org/10.1016/j.iot.2025.101869>
- [7] N. Ouyang, A. Shatte, Z. Lu, C. Chen, W. Xiang, Artificial intelligence in mitigating security threats for lightweight IoT devices: a survey of technologies, protocols, and future challenges, *IEEE Internet Things J.* 13 (7) (2026) 14096–14111. <https://doi.org/10.1109/JIOT.2025.3649316>
- [8] D. Kilichev, W. Kim, Hyperparameter optimization for 1D-CNN-Based network intrusion detection using GA and PSO, *Mathematics* 11 (17) (2023). <https://doi.org/10.3390/math11173724>
- [9] A. Abdusalomov, D. Kilichev, R. Nasimov, I. Rakhmatullayev, Y. Im Cho, Optimizing smart home intrusion detection with harmony-enhanced extra trees, *IEEE Access* 12 (2024) 117761–117786. <https://doi.org/10.1109/ACCESS.2024.3422999>
- [10] D. Kilichev, D. Turimov, W. Kim, Next-generation intrusion detection for IoT EVCS: integrating CNN, LSTM, and GRU models, *Mathematics* 12 (4) (2024). <https://doi.org/10.3390/math12040571>
- [11] F. Makhmudov, D. Kilichev, U. Giyosov, F. Akhmedov, Online machine learning for intrusion detection in electric vehicle charging systems, *Mathematics* 13 (5) (2025). <https://doi.org/10.3390/math13050712>
- [12] F. Makhmudov, G. Juraev, O. Yusupov, P. Nasriddinova, D. Kilichev, Drift-aware online ensemble learning for real-time cybersecurity in internet of medical things networks, *Mach. Learn. Knowl. Extr.* 8 (3) (2026). <https://www.mdpi.com/2504-4990/8/3/67>. <https://doi.org/10.3390/make8030067>
- [13] M. Odiahevar, W.K.G. Seah, M. Freat, A hybrid online offline system for network anomaly detection, in: 2019 28th International Conference on Computer Communication and Networks (ICCCN), 2019, pp. 1–9. <https://doi.org/10.1109/ICCCN.2019.8847011>

- [14] M. Odiathevar, W.K.G. Seah, M. Frean, A. Valera, An online offline framework for anomaly scoring and detecting new traffic in network streams, *IEEE Trans. Knowl. Data Eng.* 34 (11) (2022) 5166–5181. <https://doi.org/10.1109/TKDE.2021.3050400>
- [15] K.S. Adewole, T.T. Salau-Ibrahim, A.L. Imoize, I.D. Oladipo, M. AbdulRaheem, J.B. Awotunde, A.O. Balogun, R.M. Isiaka, T.O. Aro, Empirical analysis of data streaming and batch learning models for network intrusion detection, *Electronics* 11 (19) (2022). <https://doi.org/10.3390/electronics11193109>
- [16] E. Gelenbe, M. Nakip, Real-Time cyberattack detection with offline and online learning, in: 2023 IEEE 29th International Symposium on Local and Metropolitan Area Networks (LANMAN), 2023, pp. 1–6. <https://doi.org/10.1109/LANMAN58293.2023.10189812>
- [17] P.R. Agbedanu, S.J. Yang, R. Musabe, I. Gatare, J. Rwigema, ALMANET: a hybrid online learning IDS for real-time IoT security, *Egypt. Inform. J.* 31 (2025) 100764. <https://www.sciencedirect.com/science/article/pii/S1110866525001574>. <https://doi.org/10.1016/j.eij.2025.100764>
- [18] Z. Sun, G. Ran, Z. Jin, Intrusion detection method based on active incremental learning in industrial internet of things environment, *J. Internet Things* 4 (2) (2022) 99–111. <http://www.techscience.com/jiot/v4n2/51966>. <https://doi.org/10.32604/jiot.2022.037416>
- [19] N.W. Abderrahim, A. Benosman, Adaptive intrusion detection in IoT: combining batch and incremental learning for enhanced security, *Eng. Res. Express* 7 (1) (2025) 015278. <https://doi.org/10.1088/2631-8695/adbb3e>
- [20] Q. Lu, K. An, J. Li, J. Wang, Network intrusion detection for modern smart grids based on adaptive online incremental learning, *IEEE Trans. Smart Grid* 16 (3) (2025) 2541–2553. <https://doi.org/10.1109/TSG.2025.3535949>
- [21] F. Cerasuolo, G. Bovenzi, C. Marescalco, F. Cirillo, D. Ciuonzo, A. Pescapè, Adaptive intrusion detection systems: class incremental learning for IoT emerging threats, in: 2023 IEEE International Conference on Big Data (BigData), IEEE Computer Society, Los Alamitos, CA, USA, 2023, pp. 3547–3555. <https://doi.org/10.1109/BigData59044.2023.10386129>
- [22] D. Di Monda, A. Montieri, V. Persico, P. Voria, M. De Ieso, A. Pescapè, Few-shot class-incremental learning for network intrusion detection systems, *IEEE Open J. Commun. Soc.* 5 (2024) 6736–6757. <https://doi.org/10.1109/OJCOMS.2024.3481895>
- [23] L. Du, Z. Gu, Y. Wang, L. Wang, Y. Jia, A few-shot class-incremental learning method for network intrusion detection, *IEEE Trans. Netw. Serv. Manag.* 21 (2) (2024) 2389–2401. <https://doi.org/10.1109/TNSM.2023.3332284>
- [24] F. Cerasuolo, G. Bovenzi, D. Ciuonzo, A. Pescapè, Adaptable, incremental, and explainable network intrusion detection systems for internet of things, *Eng. Appl. Artif. Intell.* 144 (2025) 110143. <https://www.sciencedirect.com/science/article/pii/S0952197625001435>. <https://doi.org/https://doi.org/10.1016/j.engappai.2025.110143>
- [25] J. Violos, C. Krikas, P. Sarantos, A. Leivadreas, OLIDA-IDS: online learning with integrated domain adaptation for intrusion detection systems, in: GLOBECOM 2025 - 2025 IEEE Global Communications Conference, 2025, pp. 6256–6261. <https://doi.org/10.1109/GLOBECOM59602.2025.11431867>
- [26] N.F. A. H. A. Khraisat, S.I. S. P. G. Li, Adaptive memory replay for network intrusion detection: tackling data drift and catastrophic forgetting, *Comput. Netw.* 272 (2025) 111712. <https://www.sciencedirect.com/science/article/pii/S1389128625006784>. <https://doi.org/https://doi.org/10.1016/j.comnet.2025.111712>
- [27] Z. Liu, J. Xue, W. Guo, W. Du, QHCL-AOC-IDS: an uncertainty-aware online intrusion detection system with quadruplet-hard contrastive learning, in: 2025 IEEE 6th International Conference on Computer, Big Data, Artificial Intelligence (ICCBDAI), 2025, pp. 1–7. <https://doi.org/10.1109/ICCBDAI66607.2025.11388022>
- [28] C.M. Alamani, M.C. J.T. Figueroa, J.R.Q. Mundo, J.A. de Guzman, QUIC-AID: adaptive intrusion detection for QUIC traffic using online learning with ADWIN-based drift detection, in: TENCON 2025 - 2025 IEEE Region 10 Conference (TENCON), 2025, pp. 923–927. <https://doi.org/10.1109/TENCON66050.2025.11375148>
- [29] A. Kumar, V. Kumar, A.P.S. Bhaduria, A dual approach to intrusion detection in edge environments: offline and online perspectives, *SN Comput. Sci.* 7 (5) (2026) 399. <https://doi.org/10.1007/s42979-026-04978-7>
- [30] F. Alkamli, M. Derbali, T.M. Ahmed, Honey-pot-driven hybrid continuous learning platform for proactive cyber threat detection, *Adv. Artif. Intell. Mach. Learn.* 6 (2) (2026) 5176–5197. <https://doi.org/10.54364/AAIML.2026.62287>
- [31] B.R.S. Reddy, D. Ganesh, S. Kumar, A. Likhitha, B.J. Dattudu, D.V.M. Reddy, Adaptive intrusion detection enhanced by cyber threat intelligence, in: 2026 8th International Conference on Intelligent Sustainable Systems (ICISS), 2026, pp. 491–497. <https://doi.org/10.1109/ICISS67859.2026.11453603>
- [32] J. Fan, Z. Si, Z. Wang, D. Sun, J. Wu, H. Wu, OOA-UADS: offline, online, analysis-an unsupervised anomaly detection solution for multivariate time series, in: 2023 International Joint Conference on Neural Networks (IJCNN), 2023, pp. 1–9. <https://doi.org/10.1109/IJCNN54540.2023.10191780>
- [33] D. T. S. Y. A. N. S. R. Zero-day attack detection in IoT networks using graph neural networks and online learning, in: 2025 10th International Conference on Communication and Electronics Systems (ICCSES), 2025, pp. 1943–1948. <https://doi.org/10.1109/ICCSES67310.2025.11337145>
- [34] R. De Maesschalck, D. Jouan-Rimbaud, D.L. Massart, The mahalanobis distance, *Chemom. Intell. Lab. Syst.* 50 (1) (2000) 1–18. [https://doi.org/https://doi.org/10.1016/S0169-7439\(99\)00047-7](https://doi.org/https://doi.org/10.1016/S0169-7439(99)00047-7)
- [35] O. Ledoit, M. Wolf, A well-conditioned estimator for large-dimensional covariance matrices, *J. Multivar. Anal.* 88 (2) (2004) 365–411. [https://doi.org/https://doi.org/10.1016/S0047-259X\(03\)00096-4](https://doi.org/https://doi.org/10.1016/S0047-259X(03)00096-4)
- [36] R. Sebastião, J.M. Fernandes, Supporting the page-hinkley test with empirical mode decomposition for change detection, in: M. Kryszkiewicz, A. Appice, D. Ślęzak, H. Rybinski, A. Skowron, Z.W. Raś (Eds.), *Foundations of Intelligent Systems*, Springer International Publishing, Cham, 2017, pp. 492–498.
- [37] J. Aczél, Entropies, characterizations, applications and some history, in: B. Bouchon-Meunier, G. Coletti, R.R. Yager (Eds.), *Modern Information Processing*, Elsevier Science, Amsterdam, 2006, pp. 3–10. <https://doi.org/https://doi.org/10.1016/B978-0-44452075-3/50001-7>
- [38] D. Rolnick, A. Ahuja, J. Schwarz, T. Lillicrap, G. Wayne, Experience replay for continual learning, in: H. Wallach, H. Larochelle, A. Beygelzimer, F.d. Alché-Buc, E. Fox, R. Garnett (Eds.), *Advances in Neural Information Processing Systems*, Vol. 32, Curran Associates, Inc., 2019, pp. 350–360. https://proceedings.neurips.cc/paper_files/paper/2019/file/fa7cdfad1a5aaf8370ebeda47a1ffc3-Paper.pdf
- [39] G.M. van de Ven, N. Soares, D. Kudithipudi, 1.09 - Continual learning and catastrophic forgetting, in: J. Wixted (Ed.), *Learning and Memory: A Comprehensive Reference* (Third Edition), Academic Press, Oxford, 3rd ed., 2025, pp. 153–168. <https://doi.org/https://doi.org/10.1016/B978-0-443-15754-7.00073-0>
- [40] S. Dadkhah, E.C.P. Neto, R. Ferreira, R.C. Molokwu, S. Sadeghi, A.A. Ghorbani, CICIOT2024: a benchmark dataset for multi-protocol security assessment in IoMT, *Internet Things* 28 (2024) 101351. <https://www.sciencedirect.com/science/article/pii/S2542660524002920>. <https://doi.org/https://doi.org/10.1016/j.iot.2024.101351>
- [41] G. Jurman, S. Riccadonna, C. Furlanello, A comparison of MCC and CEN error measures in multi-Class prediction, *PLOS One* 7 (8) (2012) 1–8. <https://doi.org/10.1371/journal.pone.0041882>
- [42] M.T. Hossain, S.M. Meem, J.H. Setu, N. Halder, A. Islam, M.Z. Alam, Cyberattacks classification on internet of medical things using information gain feature selection and machine learning, in: 2024 Advances in Science and Engineering Technology International Conferences (ASET), 2024, pp. 1–10. <https://doi.org/10.1109/ASET670340.2024.10708692>
- [43] M. Lucia Hernandez-Jaimas, A. Martínez-Cruz, K. Alejandra Ramírez-Gutiérrez, E. Guevara-Martínez, Enhancing machine learning approach based on nlsimsa fingerprinting for ransomware detection in IoMT, *IEEE Access* 12 (2024) 153886–153897. <https://doi.org/10.1109/ACCESS.2024.3480889>
- [44] H. Naem, A. Alsirhani, F.M. Alserhani, F. Ullah, O. Krejcar, Augmenting internet of medical things security: deep ensemble integration and methodological fusion, *Comput. Model. Eng. Sci.* 141 (3) (2024) 2185–2223. <http://www.techscience.com/CMES/v141n3/58509>. <https://doi.org/10.32604/cmcs.2024.056308>
- [45] A.S. Gomez, F. Gutierrez Portela, O.A. Diaz Triana, Intrusion detection system for IoT using anomaly detection techniques, in: 2024 IEEE VII Congreso Internacional en Inteligencia Ambiental, Ingeniería de Software y Salud Electrónica y Móvil (AmITIC), 2024, pp. 1–6. <https://doi.org/10.1109/AmITIC62658.2024.10747648>